SPRITZ: A Short PRISM-Based Threshold Signature via Zero-Knowledge Proof

Hyeonhak Kim, Won Kim, and Changmin Lee*

School of Cybersecurity, Korea University, Seoul, Korea

Abstract. Threshold signatures for distributed systems require compact public keys and signatures to reduce communication overhead by avoiding packet fragmentation. However, with existing post-quantum threshold signatures, either the public key or the signature does not fit within a single unfragmented network packet.
In this work, we present SPRITZ, an isogeny-based post-quantum threshold signature scheme whose public keys and signatures both fit within a single unfragmented network packet at every NIST security level. To the best of our knowledge, SPRITZ is the first post-quantum threshold signature scheme to do so with an arbitrary number of parties. While isogeny-based signatures such as SQIsign and PRISM are known for exceptionally compact public keys and signatures, their algebraic structure makes thresholdization for an arbitrary number of parties highly nontrivial. We address this challenge by introducing a novel graph-based threshold access structure tailored to the isogeny setting. For security against known attacks, we employ a non-interactive zero-knowledge (NIZK) protocol for the verification of isogeny push-forward procedures.
At NIST security levels I/III/V, SPRITZ achieves public keys of 129/193/257 bytes and signatures of 223/335/447 bytes, respectively. Among the schemes submitted to the NIST MPTC (Multi-Party Threshold Cryptography) previews phase 2 whose public keys fit within a single unfragmented network packet, SPRITZ achieves the smallest signature size. We also provide a proof-of-concept implementation of SPRITZ.

1 Introduction

Distributed systems have become a core component of the infrastructure for security-critical applications [1], as they can continue to operate despite node failures, network disruptions, or partial compromises. In such settings, placing a cryptographic secret key in the hands of a single party creates an inherent bottleneck: compromising or disconnecting that party may be enough to break security or halt the system. Threshold cryptography [2] addresses this issue by distributing a secret capability among n parties so that any set of at least t parties can jointly perform the intended cryptographic operation, while smaller coalitions cannot. Among threshold primitives, threshold signatures [3] are especially important, preserving the usual verification interface while distributing trust across multiple participants [4].

* Corresponding Author

Compactness is a key efficiency requirement for threshold signatures. In practical distributed deployments, it is highly desirable that the signature and the public key fit within a single unfragmented network packet, since if either artifact exceeds the packet size then it must be fragmented or carried across multiple packets. In that regime, even a single packet loss can stall the entire operation and increase latency [5]. This issue is especially acute for signatures [6], which lie on the critical path of message dissemination, and for public keys, which must be distributed to verifiers and replicated across the system. Consequently, an efficient threshold signature scheme should keep both the signature and the public key compact enough to fit within a single unfragmented packet.

Many post-quantum digital signature schemes achieve quantum resistance at the cost of large verification artifacts: for example, standardized lattice-based signatures such as ML-DSA have kilobyte-scale public keys and signatures [7], whereas hash-based signatures such as SLH-DSA keep public keys compact but produce signatures of several to tens of kilobytes [8]. Thresholdizing these signature schemes does not reduce the size of the public key or the signature; moreover, some threshold signature schemes have larger public keys or signatures than those of the underlying non-threshold schemes. Thus, large public keys or large signatures remain a central obstacle for post-quantum threshold signatures.

Isogeny-based signatures on supersingular elliptic curves defined over $\mathbb{F}_{p^2}$, such as SQIsign [9] and PRISM [10], provide a markedly different design point among post-quantum signatures. Both their public keys and signatures are extremely compact when compared with other post-quantum signature schemes. This makes them particularly attractive as a foundation for compact post-quantum threshold signatures. However, such schemes are substantially more difficult to thresholdize than other post-quantum signature schemes: the signing secret is not a linear scalar but a highly structured isogeny-related object, and the signing procedure involves non-linear algebraic operations that do not naturally interact with standard secret-sharing techniques. As a result, despite the appealing size profile, constructing threshold signatures for an arbitrary number of parties from SQIsign and PRISM remains open. [1]

1.1 Our contributions

Our contributions are as follows:

– We introduce SPRITZ, a PRISM-based threshold signature scheme that, to the best of our knowledge, is the first threshold PRISM construction supporting an arbitrary number of parties. We further construct an adaptive variant of SPRITZ in which parties may participate in an arbitrary order. We also provide a proof-of-concept implementation of SPRITZ.

[1] There was a presentation on threshold PRISM at the NIST Workshop on Multi-Party Threshold Schemes 2026 [11]. According to their slides, it supports (n, t)-threshold signatures only for $1 \leq t \leq n \leq 32$.

– SPRITZ achieves the most compact signature sizes among the post-quantum threshold signatures submitted to the NIST MPTC previews phase 2 [12] whose public keys fit within a single unfragmented network packet[2]. Moreover, SPRITZ is the only scheme in this comparison class whose public key and signature both fit within a single unfragmented network packet at every NIST-I/III/V security level. This comparison is shown in Table 1.

Table 1: Public key (PK) and signature (Sig) sizes of post-quantum signatures submitted to NIST MPTC previews phase 2 in bytes at NIST security levels I/III/V.

	PK (bytes)			Sig (bytes)		
Scheme	I	III	V	I	III	V
Ours	129	193	257	223	335	447
LMS(Haystack) [13]	–	48	56	–	≥ 1140	≥ 1772
XMSS(Haystack) [13]	–	52	68	–	≥ 1732	≥ 2820
Hermine [14]	1920	2560	3200	4832	6448	8064
LEAST [15]	≥ 13940	≥ 35074	≥ 65793	≥ 1440	≥ 4896	≥ 8960
Mithril [16]	1312	1952	2592	2420	3309	4627
Quorus [17]	1312	1952	2592	2420	3309	4627
Trilithium [18]	1312	1952	2592	2420	3309	4627
Traccoon(Tanuki) [19]	3900	5800	7200	12700	18900	21600
UOV(Vinaigrette) [20]	≥ 43576	189232	446992	≥ 96	200	260
MAYO(Vinaigrette) [20,21]	1420 | 4912	2986	5554	454 | 186	681	964
Macaw(PQarrots) [22]	128 | 65536	–	–	1616 | 400	–	–
Lemur [23]	≥ 185000	–	–	$\geq 2752n$	–	–
LoTRS [24]	≥ 24474	–	–	$7296nt$	–	–
TALUS [25]	1312	1952	2592	2420	3309	4627

For Haystack, public key and signature sizes are not provided so we have computed them from RFC documents of LMS/XMSS [26,27].
For MAYO, entries of the form $x \mid y$ correspond to MAYO1 and MAYO2, respectively. For Macaw, they correspond to the optimal-parameter instances minimizing the public key and the signature, respectively.

– We design the new LongIdealToIsogeny algorithm which translates a long smooth ideal to a set of kernel-representable smooth isogenies using the Qlapoti algorithm. This algorithm is used to generate various isogeny-based graph structures used in our threshold access structure. The algorithm first decomposes a long ideal into a sequence of smaller-norm ideals, and translates all small ideals to the corresponding isogenies.

1.2 Technical overview

High-level idea of thresholdizing PRISM. At a high level, we thresholdize the PRISM signature scheme by decomposing its secret key isogeny path $\phi_{\mathsf{sk}} : E_0 \to E_t$ between elliptic curves into each party i's partial isogeny path ϕ_{sk_i}

[2] Here, "single unfragmented network packet" informally refers to roughly the usual 1.2–1.5 KB payload budget on common Internet paths.

such that $\phi_{\mathsf{sk}_{i_t}} \circ \cdots \circ \phi_{\mathsf{sk}_{i_1}} : E_0 \to E_t$ can be a valid secret key for any pairwise distinct $1 \leq i_1, \ldots, i_t \leq n$. Dividing a secret key isogeny path into a single chain of t paths works for constructing a multi-signature scheme. The core idea is to generate other isogeny paths, all with the same start curve E_0 and end curve E_t, and to distribute their edges among n parties so that any t parties can recover at least one complete path. This naturally leads to a graph-based threshold access structure: vertices represent intermediate curves, edges represent secret isogeny steps, and an authorized set is one whose assigned edges contain a directed path from E_0 to E_t. Our high-level idea is illustrated in Figure 1.

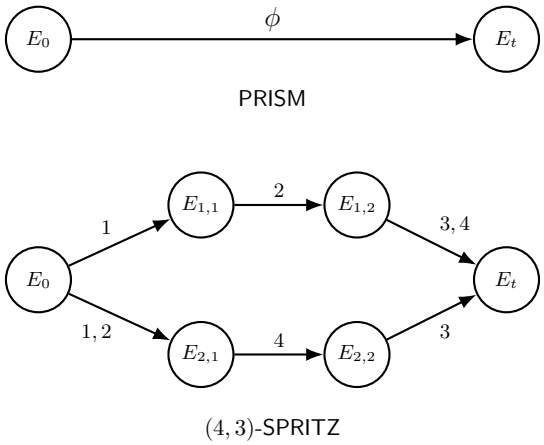


Fig. 1: High-level illustration of PRISM and SPRITZ. The bottom graph illustrates SPRITZ for the case $(n, t) = (4, 3)$. The edge labels indicate the parties holding the corresponding edges.

Construction of isogeny graphs using LongIdealToIsogeny. To construct the isogeny graph illustrated in Figure 1, we first build its counterpart in the quaternion setting, where long paths are represented by ideals of large smooth norm. We then translate these ideals into the corresponding isogeny paths under the Deuring correspondence. Our LongIdealToIsogeny algorithm performs this translation by decomposing each ideal into smaller-norm factors and computing a kernel representation for each corresponding isogeny. The key idea is to use a variant of Qlapoti [28] to construct auxiliary isogenies from the starting curve E_0 to the intermediate curves along the path. The explicitly known endomorphism ring of E_0 allows us to compute the relevant kernel subgroups on E_0, which we then transport to the intermediate curves via these auxiliary isogenies. This yields the kernel-represented isogeny edges needed to realize the desired graph.

Efficient zero-knowledge proofs of push-forwards. Directly proving the full push-forward relation requires checking equality between isogenies whose degrees contain large prime factors, making this relation costly to encode in rank-1 constraint system (R1CS). To reduce this cost, we introduce a weak relation

$\mathcal{R}_{\mathsf{weak\text{-}push}}$ in Definition 3 that replaces isogeny equality with comparisons of the corresponding curves. Specifically, it checks that the claimed output has the expected intermediate curve, reached after its degree-q factor, and final codomain, by comparing their j-invariants. Although these checks do not imply the full push-forward relation in general, we provide heuristic evidence that, under our parameter choices and the required degree and validity checks, they identify the correct push-forward with overwhelming probability, and formulate this claim as an explicit assumption (Definition 4). The witness-dependent isogeny computations then reduce to two smooth degree-3^b isogeny codomain computations, while checks involving only public data are performed outside the circuit. This yields a compact R1CS instance and enables efficient NIZK proof generation and verification.

2 Preliminaries

In this section, we introduce the basic notions and background necessary to understand this paper.

Throughout the paper, we use the following notation. For $n \in \mathbb{N}$, let $[n] := \{1, \ldots, n\}$. For an elliptic curve E, 0_E denotes its identity element, $E(F)$ denotes the set of F-rational points of E, and $E[m]$ denotes its m-torsion subgroup. We let $p \equiv 3 \bmod 4$ prime, $E_0 : y^2 = x^3 + x$ denote the supersingular elliptic curve defined over $\mathbb{F}_{p^2}$. For an isogeny ϕ, we write $\deg(\phi)$, $\mathrm{dom}(\phi)$, $\mathrm{Im}(\phi)$, and $\widehat{\phi}$ for its degree, domain curve, image curve, and dual isogeny, respectively. For a point P, $[m]P$ denotes scalar multiplication of P by m. We write $[\phi]_*\psi$ and $[\phi]^*\psi$ for the push-forward and pull-back of ψ along ϕ, respectively. Finally, $\mathcal{O}_0$ denotes a maximal order isomorphic to $\mathrm{End}(E_0)$, $\mathrm{nrd}(I)$ denotes the reduced norm of an ideal I, and H_{prime} denotes the hash-to-primes map of PRISM.

We also explain fundamental concepts and theorems about isogenies and quaternion algebras necessary for understanding this paper in Appendix A.1. The PRISM signature is explained in Appendix A.4.

2.1 Threshold Signatures

Informally, a threshold signature protocol distributes a signing capability across n parties so that (i) any subset of at least t parties can jointly produce a *single* standard signature verifiable under one public verification key, while (ii) any coalition of fewer than t parties cannot produce signatures on its own. This eliminates a single point of compromise for the signing key and improves availability, since signing can proceed as long as at least t parties are online.

Definition 1 (Threshold Signature Scheme). *Let λ be a security parameter, and let $R, n \in \mathbb{N}$ and $t \in [n]$, where R is the number of rounds, n is the total number of parties, and t is the threshold. An* R-round (n, t)-threshold signature scheme *is a tuple of algorithms*

$$\mathsf{TSS} \;=\; \big(\mathsf{Setup}, \mathsf{KeyGen}, (\mathsf{Sign}_r)_{r\in[R]}, \mathsf{Combine}, \mathsf{Verify}\big),$$

with the following syntax. Let $S \subseteq [n]$ denote the set of signers (the active set*).*

- $\mathsf{Setup}(\lambda, n, t) \to \mathsf{pp}$. *It outputs public parameters* pp.
- $\mathsf{KeyGen}(\mathsf{pp}) \to \big(\mathsf{pk}, (\mathsf{sk}_i)_{i\in[n]}, (\mathsf{pk}_i)_{i\in[n]}\big)$. *It outputs a public key* pk*, partial signing-key shares* $(\mathsf{sk}_i)_{i\in[n]}$ *and partial public keys* $(\mathsf{pk}_i)_{i\in[n]}$.
- $\mathsf{Sign}_r\big(\mathsf{pk}, S, (\mathsf{pk}_j)_{j\in S}, \mathsf{msg}, (\mathsf{pm}_{r-1}^j)_{j\in S}, i, \mathsf{sk}_i\big) \to \mathsf{pm}_r^i$ *for each* $r \in [R]$. *It outputs the round-*r *protocol message* pm_r^i. *We set* $\mathsf{pm}_0^i := \perp$ *for all* $i \in [n]$ *as the input to* Sign_1.
- $\mathsf{Combine}\big(\mathsf{pk}, S, \mathsf{msg}, (\mathsf{pm}_r^i)_{r\in[R],\, i\in S}\big) \to \mathsf{sig}$. *It outputs a signature* sig.
- $\mathsf{Verify}(\mathsf{pk}, \mathsf{msg}, \mathsf{sig}) \to \{0,1\}$. *It outputs* 1 *if and only if* sig *is valid for* msg *under* pk.

A threshold signature scheme also satisfies the properties specified in Appendix A.2.

2.2 Non-Interactive Zero-Knowledge Proofs

Let $\lambda \in \mathbb{N}$ denote the security parameter, and let $R = \{R_\lambda\}_{\lambda\in\mathbb{N}}$ be a polynomial-time decidable relation with polynomially bounded instance and witness lengths. The language associated with R is

$$L_{R,\lambda} := \{x \mid \exists w :\ R_\lambda(x, w) = 1\}.$$

We work in the quantum random oracle model (QROM). Throughout the following definitions, H denotes the proof-system random oracle $H : \{0,1\}^* \to \{0,1\}^{\kappa(\lambda)}$, where κ is a polynomially bounded function.

Quantum adversaries may access H in superposition through $U_H : |u, z\rangle \mapsto |u, z \oplus H(u)\rangle$. All query lengths and the number of queries are polynomially bounded.

Definition 2 (Non-interactive zero-knowledge proof in the QROM). *A non-interactive zero knowledge proof system (NIZK) for R consists of probabilistic polynomial-time algorithms*

$$\Pi = \big(\mathsf{Prove}, \mathsf{Verify}\big)$$

with the following syntax:

$$\pi \leftarrow \mathsf{Prove}^H(1^\lambda, x, w),$$
$$b \leftarrow \mathsf{Verify}^H(x, \pi) \in \{0,1\}.$$

Here, x is a statement, w is a witness satisfying $R_\lambda(x, w) = 1$, and π is the proof. Both algorithms use the same oracle H.

The non-interactive zero-knowledge proof protocol also satisfies the properties described in Appendix A.3.

3 Construction of SPRITZ

In this section, we construct the threshold signature scheme SPRITZ using several equivalent isogeny paths in the PRISM setting.

For the threshold construction, we use the prime $p = 2^a 3^b f - 1$ where f is a small cofactor satisfying $p \equiv 3 \bmod 4$, and the following prime set

$$\mathsf{Primes}_a^{\mathsf{TS}} := \{q : q \text{ and } 2^a - q \text{ are prime, } 2^{a-1} < q < 3 \cdot 2^{a-2}\},$$

for the security reason discussed in Appendix E.2.2.

Our scheme also requires a family $\mathcal{H}$ of functions $h_i : [n] \to [t]$ such that, for every subset $S \subseteq [n]$ of size t, there exists $h_i \in \mathcal{H}$ such that $h_i(S) = [t]$. Such a family of functions is called an (n, t)-*perfect hash family*. Its definition and optimal constructions are explained in Appendix B.1.

In order to use an (n, t)-perfect hash family, we need two efficient algorithms: GenHashFamily (optimal construction algorithm based on Theorem 3 or Appendix C.1) that generates an (n, t)-perfect hash family and SelectHashFunc (Algorithm 8) that finds a suitable function h_i such that $h_i(S) = [t]$ for the given subset of size t. The expected time complexity is $O(\sqrt{t} \cdot e^t \log n)$ according to Lemma 2, or $\binom{n-\lceil \frac{t}{2} \rceil}{\lfloor \frac{t}{2} \rfloor}$ according to Lemma 3. The Setup algorithm for our threshold signature scheme is described in Algorithm 1.

Algorithm 1: $\mathsf{Setup}(\lambda, n, t)$

Input: A security parameter λ and positive integers n, t such that $2 \leq t \leq n$

Output: A prime p, an (n, t)-perfect hash family $\mathcal{H}$, and the common reference string $\mathsf{crs}_{\mathsf{init}}$

1 Compute $\mathcal{H} \leftarrow \mathsf{GenHashFamily}(n, t)$.
2 Compute a prime p according to Appendix C.5.
3 Sample a random string $\mathsf{crs}_{\mathsf{init}}$ for InitSig.
4 **return** $(p, \mathcal{H}, \mathsf{crs}_{\mathsf{init}})$.

3.1 Key generation

The key generation algorithm is performed by a trusted dealer. Let p be a prime of the form $p = 2^a 3^b f - 1$, where $a \approx 2\lambda$ and $3^b \approx 2^{2\lambda}$ for the security parameter λ. To prevent the meet-in-the-middle (MITM) attack, we have the following constraint: $3^b > 2^{2\lambda}$.

The target curve E_t is constructed by sampling a random left $\mathcal{O}_0$-ideal I and computing the codomain curve corresponding to its right order. According to the results of Section B.1, we need to generate $O(e^t \log n)$ equivalent paths of degree 3^{bt}. To obtain multiple distinct equivalent paths of this degree, we employ the KLPT algorithm [29], which takes as input a left $\mathcal{O}_0$-ideal I and outputs an equivalent ideal of smooth norm. For more details, see Appendix C.2.

In line 1 of Algorithm 10, we choose e such that $3^e > p/\mathrm{nrd}(I)$ in order to ensure that FullRepresentInteger succeeds. Moreover, StrongApproximation succeeds

only if $3^{bt-e} > p \cdot \mathrm{nrd}(I)^3$ (see Section 5.3 in [9]). Combining these conditions yields the constraint $3^{bt} > p^2 \cdot \mathrm{nrd}(I)^2$. In particular, when $\mathrm{nrd}(I) \approx p^{1/4}$, this implies that Algorithm 10 requires $bt \geq \lceil 5/2 \cdot \log_3(p) \rceil$. This leads to the combined constraint $t \geq \frac{1}{b} \left\lceil \frac{5}{2} \cdot \log_3(p) \right\rceil \approx 5$.

After obtaining an ideal J of norm 3^{bt} from Algorithm 10, it must be converted into the corresponding isogeny to serve as a partial secret. The main idea is to decompose J as a product $J = J_1 \times \cdots \times J_t$, where $\mathrm{nrd}(J_i) = 3^b$ for each i, and then compute kernel representations for each component J_i.

Algorithm 2: $\mathsf{LongIdealToIsogeny}(J)$

Input: A left $\mathcal{O}_0$-ideal J with $\mathrm{nrd}(J) = 3^{bt}$ such that $E_0[J]$ is cyclic.
Output: The kernel representations of φ_i's corresponding to J_i's, where $J = J_1 \cdots J_t$ such that $\mathrm{nrd}(J_i) = 3^b$.

Compute $\varphi_1 \leftarrow \mathsf{IdealToIsogeny}(J_1)$. // using Qlapoti algorithm, Appendix C.3
Set $\mathcal{T} \leftarrow \{\varphi_1\}$.
for $\mathsf{step} \leftarrow 1 \ldots t-1$ **do**
  Compute $L \leftarrow \mathsf{EquivalentPrimeIdeal}(J + 3^{\mathsf{step} \cdot b} \cdot \mathcal{O}_0)$.
  Compute $\psi_L \leftarrow \mathsf{IdealToIsogeny}(L)$. // using Qlapoti algorithm, Appendix C.3
  Compute the 3^b-torsion cyclic subgroup $G = E_0[L \cap J'] \cap E_0[3^b]$, where $J' = (J + 3^{\mathsf{step} \cdot b} \cdot \mathcal{O}_0)^{-1} \cdot (J + 3^{(\mathsf{step}+1)b} \cdot \mathcal{O}_0)$.
  Compute $\varphi_{\mathsf{step}+1}$ such that $\ker \varphi_{\mathsf{step}+1} = \psi_L(G)$.
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\varphi_{\mathsf{step}+1}\}$.
return the kernel representations of $\mathcal{T} = \{\varphi_i\}_{1 \leq i \leq t}$.

Algorithm 3: $\mathsf{KeyGen}(p, n, t, \mathcal{H})$

Input: A base prime p such that $p \equiv 3 \mod 4$, positive integers n and t such that $5 \leq t \leq n$, and an (n,t)-perfect hash family $\mathcal{H}$.
Output: A public key pk, partial secret keys sk_i, and partial public keys pk_i for $1 \leq i \leq n$.

Choose a random left $\mathcal{O}_0$-ideal I such that $\mathrm{nrd}(I) \approx p^{1/4}$.
for $i \leftarrow 1$ *to* N **do**
  $J_i \leftarrow \mathsf{KLPT}_3(I, bt)$. // Algorithm 10
  Compute $\{\varphi_{i,j}\}_{j \in [t]} \leftarrow \mathsf{LongIdealToIsogeny}(J_i)$. // Algorithm 2
For every $(i,j) \in [N] \times [t]$, compute $\mathsf{Com}_{i,j} \leftarrow H_{\mathsf{key}}(\mu_{i,j})$, where $\ker(\varphi_{i,j}) = \langle R_{i,j} + [\mu_{i,j}] S_{i,j} \rangle$ for the canonical 3^b-torsion basis $(R_{i,j}, S_{i,j})$ of the domain curve of $\varphi_{i,j}$.
Set $\mathsf{pk} \leftarrow E_{1,t}$, where $E_{i,0} = E_0$ and $E_{i,j} = \mathrm{Im}(\varphi_{i,j})$.
for $i \leftarrow 1$ *to* n **do**
  Set $\mathsf{pk}_i \leftarrow ((\mathsf{Com}_{1,h_1(i)}, E_{1,h_1(i)}), \ldots, (\mathsf{Com}_{N,h_N(i)}, E_{j,h_j(i)}))$.
  Set $\mathsf{sk}_i \leftarrow (\varphi_{1,h_1(i)}, \ldots, \varphi_{N,h_N(i)})$.
return $\{\mathsf{pk}_i\}_{1 \leq i \leq n}, \mathsf{pk}, \{\mathsf{sk}_i\}_{1 \leq i \leq n}$.

At the i-th step, we compute an ideal L equivalent to $J_1 \times \cdots \times J_{i-1}$ such that $\mathrm{nrd}(L)$ is coprime to 3. We then compute the corresponding isogeny $\psi_L : E_0 \to E_i$ associated to L using the Qlapoti algorithm [28]. Finally, we evaluate the kernel of the ideal $[L]^* J_i$ on E_0 via the isogeny ψ_L, which yields the kernel representation of J_i on E_i. This procedure is described in Algorithm 2.

Algorithm 2 uses EquivalentPrimeIdeal [9] as a subroutine. EquivalentPrimeIdeal takes as input a left $\mathcal{O}_0$-ideal and outputs its equivalent left $\mathcal{O}_0$-ideal of prime norm by performing a randomized search over elements of the input ideal.

Using GenHashFamily, KLPT, and LongIdealToIsogeny as subroutines, the key generation procedure is described in Algorithm 3.

3.2 Signing and verification

The parties generate a signature sequentially according to the predetermined order. When t parties gather, they deterministically select a hash function $h_i \in \mathcal{H}$ such that $h_i|_S$ is surjective, where S denotes the set of indices of the participating parties. According to the ordering induced by h_i, the parties compute partial signatures sequentially.

The first party, starting from the special curve E_0 with known endomorphism ring, generates an isogeny $\sigma_0 : E_0 \to E_0'$ of degree $q(2^a - q)$, where $q = H_{\mathsf{prime}}(\mathsf{pk} \| m)$ (see Appendix C.4) and m is the message. Each j-th party then computes the push-forward

$$\sigma_j = [\varphi_{i,j}]_* \sigma_{j-1}$$

using its partial secret isogeny $\varphi_{i,j}$. As a result, the final party obtains $\sigma_t = [\varphi_{i,t}]_* \sigma_{t-1} : E_t \to E_t'$, which is an isogeny of degree $q(2^a - q)$. It becomes the signature of the message m as in PRISM [10].

However, in a naive application, each honest party can be exploited by an adversary as a push-forward oracle, leading to the key-recovery attack proposed by Maino and Petit [30]. They show that an attacker can recover the secret isogeny $\varphi : E_1 \to E_2$ of the push-forward oracle, provided that the attacker knows the endomorphism ring of the starting curve E_1.

In their attack, the adversary constructs two input isogenies $\phi_1 : E_1 \to F_1$ and $\phi_2 : E_1 \to F_2$ of degree $q(2^a - q)$, such that there exists a small-degree isogeny $\epsilon : F_1 \to F_2$ for which $\theta = \widehat{\phi}_2 \circ \epsilon \circ \phi_1$ is an endomorphism of E_1. The isogenies are chosen so that $\theta(\ker \varphi) = \ker \varphi$, and hence the push-forward $[\varphi]_* \theta$ remains an endomorphism of E_2. The pair (ϕ_1, ϕ_2) is constructed using the known endomorphism ring of E_1. Given the two push-forwards $\big([\varphi]_* \phi_1,\ [\varphi]_* \phi_2\big)$, the attacker can recover $[\varphi]_* \epsilon$ by a brute-force search over the small-degree isogenies between the corresponding codomain curves, and thereby obtain the endomorphism $[\varphi]_* \theta$. Moreover, for arbitrary integers u, v, the attacker can construct $[\varphi]_* \big([u]\theta + [v]\big)$. When the degree of $[u]\theta + [v]$ is sufficiently smooth, this enables the attacker to extract torsion information and eventually recover the secret isogeny.

To prevent this attack, we introduce the following two countermeasures:

1. The isogeny σ_0 is generated deterministically from the common reference string.
2. Every push-forward computation is accompanied by a non-interactive zero-knowledge proof verifying that the output was computed correctly.

The first countermeasure prevents an adversary from controlling the input isogenies given to the push-forward oracles. The second prevents corrupted parties from returning maliciously chosen isogenies instead of the correctly computed push-forward results. To implement the second countermeasure efficiently, we therefore need an efficient way to prove the correctness of the push-forward relation.

Zero-knowledge proof of correct push-forward

Definition 3 (Weak Relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ for push-forwards). *We define the relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ as follows. A pair of an instance and a witness $(x = (C, q, \sigma_{\mathsf{prev}}, \sigma), w = \mu) \in \mathcal{R}_{\mathsf{weak\text{-}push}}$ if and only if all isogeny representations below are valid and cyclic, and*

$$\begin{aligned} C &= H_{\mathsf{key}}(\mu), \\ \mathrm{Im}(\sigma_1) \cong \mathrm{Im}(\varphi'), \ker \varphi' &= \langle \sigma_{\mathsf{prev},1}(R) + [\mu]\sigma_{\mathsf{prev},1}(S) \rangle, \\ \mathrm{Im}(\sigma) \cong \mathrm{Im}(\varphi''), \ker \varphi'' &= \langle \sigma_{\mathsf{prev}}(R) + [\mu]\sigma_{\mathsf{prev}}(S) \rangle \end{aligned}$$

where (R, S) is a canonical 3^b-torsion basis of the domain curve of σ_{prev}, and $\sigma_{\mathsf{prev},1}$ and σ_1 are the degree-q factors canonically determined by $\ker \sigma_{\mathsf{prev}}[q]$ and $\ker \sigma[q]$, respectively. Also let $\mathrm{dom}(\sigma) \cong \mathrm{Im}(\varphi)$, where $\ker \varphi = \langle R + [\mu]S \rangle$. Note that this additional condition can be publicly verified, since $\mathrm{Im}(\varphi)$ is publicly provided in SPRITZ.

The relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ does not rigorously verify the push-forward relation such as

$$\sigma \stackrel{?}{=} [\varphi]_* \sigma_{\mathsf{prev}},$$

where $\ker \varphi = \langle R + [\mu]S \rangle$. Instead, this relation only checks that

$$\begin{aligned} \ker(\varphi) &\stackrel{?}{=} \langle R + [\mu]S \rangle, \\ \mathrm{Im}([\sigma_{\mathsf{prev},1}]_* \varphi) &\stackrel{?}{\cong} \mathrm{Im}(\sigma_1), \text{ and} \\ \mathrm{Im}([\sigma_{\mathsf{prev}}]_* \varphi) &\stackrel{?}{\cong} \mathrm{Im}(\sigma). \end{aligned}$$

In general, these codomain checks do not imply the full push-forward equality. However, it is a reasonable heuristic by the following reason.

Suppose that σ is chosen uniformly at random among isogenies of the prescribed degree, and that its degree is verified. Let $S_{a,q}(\varphi) = \{\sigma : \deg(\sigma) = q(2^a - q) \text{ and } \mathrm{dom}(\sigma) \cong \mathrm{Im}(\varphi)\}$. The cardinality of $S_{a,q}(\varphi)$ is equal to the number of cyclic subgroups of order $q(2^a - q)$, namely $|S_{a,q}(\varphi)| = (q+1)(2^a - q + 1) \approx$

$2^{2a} \approx p$. Since there are approximately $p/12$ supersingular elliptic curves over $\mathbb{F}_{p^2}$, the expected number of isogenies in $S_{a,q}(\varphi)$ having a fixed codomain is approximately 12. More precisely, for $T_{a,q}(\varphi, E) = \{\sigma \in S_{a,q}(\varphi) : \mathrm{Im}(\sigma) \cong E\}$, we heuristically expect $|T_{a,q}(\varphi, E)| \approx 12$. Now write $\sigma = \sigma_2 \circ \sigma_1, \deg(\sigma_1) = q, \deg(\sigma_2) = 2^a - q$. There are $q + 1 \approx \sqrt{p}$ possible codomain curves for σ_1. Hence, when $p \approx 2^{4\lambda}$, for $U_{a,q}(\varphi, E_1, E_2) = \{\sigma \in T_{a,q}(\varphi, E_2) : \mathrm{Im}(\sigma_1) \cong E_1\}$, the expected cardinality is

$$|U_{a,q}(\varphi, E_1, E_2)| \approx 12 \cdot \frac{12\sqrt{p}}{p} = \frac{144}{\sqrt{p}} = \Theta(2^{-2\lambda}).$$

Now, under the Ramanujan property of supersingular isogeny graphs, we assume that the cardinality $|U_{a,q}(\varphi, E_1, E_2)|$ is well-distributed with respect to (E_1, E_2) when q and 2^a are large enough. That is, when we choose (E_1, E_2) uniformly at random, we expect to meet one isogeny in $U_{a,q}(\varphi, E_1, E_2)$ with probability $\Theta(2^{-2\lambda})$.

Thus, except with negligible probability, there is at most one isogeny σ consistent with the prescribed intermediate and codomain curves. This supports the heuristic claim that the relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ uniquely and correctly determines σ. We therefore introduce the following heuristic assumption.

Definition 4 (Heuristic assumption). *Let* $\ker \varphi = \langle R + [\mu]S \rangle$ *and* $C = H_{\mathsf{key}}(\mu)$. *Then, for* $((C, q, \sigma_{\mathsf{prev}}, \sigma), \mu) \in \mathcal{R}_{\mathsf{weak\text{-}push}}$, *it satisfies the equality*

$$\sigma = [\varphi]_* \sigma_{\mathsf{prev}},$$

with overwhelming probability $1 - \Theta(2^{-2\lambda})$.

We now need to ensure that σ is uniformly distributed. The distribution of σ depends on that of σ_{prev}. In our threshold protocol, however, σ_{prev} is itself the output of the preceding push-forward computation. Therefore, it suffices to show that uniformity is preserved by the push-forward operation. The following lemma establishes that a uniformly distributed input isogeny induces a uniformly distributed output isogeny.

Lemma 1 (Distribution preservation under push-forward). *Let* $D = q(2^a - q)$ *and let* $\varphi : E_1 \to E_2$ *have degree coprime to* D. *Pushing forward by* φ *maps a uniformly sampled cyclic* D*-isogeny from* E_1 *to a uniformly sampled cyclic* D*-isogeny from* E_2.

Proof. The restriction $\varphi|_{E_1[D]} : E_1[D] \to E_2[D]$ is an isomorphism. It therefore induces a bijection $\mathrm{Cyc}_D(E_1) \to \mathrm{Cyc}_D(E_2)$ by $K \mapsto \varphi(K)$, where $\mathrm{Cyc}_D(E)$ is the set of cyclic subgroups of order D in $E[D]$. A bijection preserves the uniform distribution on kernels, and a cyclic isogeny is determined by its kernel up to codomain isomorphism. □

By Lemma 1, it remains to sample the initial isogeny σ_0 from the uniform distribution. The distribution is obtained from Algorithm 4 shown below.

Concrete construction of the signing algorithm Algorithm 4 generates the initial isogeny σ_0 from the uniform distribution heuristically. We use the same heuristic assumption as [31], which states that the output distribution of the $\mathsf{FullRepresentInteger}$ algorithm is uniform.

Algorithm 4: (Heuristic) $\mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$

Input: An independent public random string $\mathsf{crs}_{\mathsf{init}}$, a prime number $q \in \mathsf{Primes}_a^{\mathsf{TS}}$.

Output: A kernel representation of $(2^a, 2^a)$-isogeny Φ.

Compute an endomorphism $\theta \leftarrow \mathsf{FullRepresentInteger}_{E_0}(q(2^a - q) \cdot 3^b; \mathsf{crs}_{\mathsf{init}})$.
Compute $P', Q' = \theta(P_0), \theta(Q_0)$.
Let $K = \ker(\widehat{\theta}) \cap E_0[3^b]$.
Compute $\eta : E_0 \to E_A$ with kernel K.
Compute $P_A, Q_A = [3^{-b}]\eta(P'), [3^{-b}]\eta(Q')$.
Compute Φ with kernel $\langle([-q]P_0, P_A), ([-q]Q_0, Q_A)\rangle$.
return $(E_0, ([-q]P_0, [-q]Q_0), E_A, (P_A, Q_A))$ as a representation of Φ.

Algorithm 5: $\mathsf{PushForwardSig}(\sigma_{\mathsf{prev}}, \varphi)$

Input: A partial signature $\sigma_{\mathsf{prev}} : E_1 \to E_1'$ of degree $q(2^a - q)$ in the two-dimensional representation, and a 3^b-isogeny $\varphi : E_1 \to E_2$.

Output: The push-forward partial signature $\sigma = [\varphi]_*\sigma_{\mathsf{prev}} : E_2 \to E_2'$ in the two-dimensional representation.

Parse the two-dimensional representation of σ_{prev} as $(E_1, P_1, Q_1, E_1', P_1', Q_1')$.
Compute $\varphi' = [\sigma_{\mathsf{prev}}]_*\varphi : E_1' \to E_2'$ by evaluating $\ker \varphi' = \sigma_{\mathsf{prev}}(\ker \varphi)$.
Compute $P_2, Q_2 \leftarrow \varphi(P_1), \varphi(Q_1),\ P_2', Q_2' \leftarrow \varphi'(P_1'), \varphi'(Q_1')$.
Set σ as an isogeny embedded in the two-dimensional representation $(E_2, P_2, Q_2, E_2', P_2', Q_2')$.
Compute a canonical 2^a-torsion basis P_{can}, Q_{can} on the curve E_2.
Compute $P_{can}', Q_{can}' \leftarrow \sigma(P_{can}), \sigma(Q_{can})$.
return $(E_2, [-q]_{E_2}P_{can}, [-q]_{E_2}Q_{can}, E_2', P_{can}', Q_{can}')$ as a two-dimensional representation of σ.

In Algorithm 4, $\mathsf{FullRepresentInteger}_{E_0}(*; \mathsf{crs}_{\mathsf{init}})$ denotes running the probabilistic algorithm with $\mathsf{crs}_{\mathsf{init}}$ as its public random tape.

Algorithm 5 takes as input an isogeny σ_{prev} and computes its push-forward $\sigma = [\varphi]_*\sigma_{\mathsf{prev}}$ using the secret isogeny φ.

Using Algorithm 4 and Algorithm 5 as subroutines, the concrete signing procedure is described in Algorithm 6. In the signing algorithm, we use a NIZK protocol that satisfies the properties of Definition 7 with H_{nizk} as its random oracle.

Algorithm 6: $\mathsf{Sign}(\mathsf{PK}, S, m, \Sigma_{\mathsf{prev}}, k, \mathsf{sk}_k)$

Input: A tuple of public keys $\mathsf{PK} = (\{\mathsf{pk}_i\}_{i\in S}, \mathsf{pk})$, an (n,t)-perfect hash family $\mathcal{H}$, the common reference string $\mathsf{crs}_{\mathsf{init}}$, a signing subset $S \subseteq [n]$ with $|S| = t$, a message m, a set of previous partial signatures and proofs Σ_{prev}, the signer index $k \in S$, and the partial secret key $\mathsf{sk}_k = \left(\varphi_{1,h_1(k)}, \ldots, \varphi_{N,h_N(k)}\right)$.

Output: The set of partial signatures $\Sigma_k = \left(\{\sigma_i\}_{0\le i\le r}, k, \{\pi_i^{\mathsf{sig}}\}_{1\le i\le r}\right)$.

Subroutine: A non-interactive zero knowledge protocol $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{weak\text{-}push}}}$ for the relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$.

Compute $(h_\ell, \ell) \leftarrow \mathsf{SelectHashFunc}_{n,t}(\mathcal{H}, S)$.
Parse $\{\mathsf{Com}_{j,h_j(i)}\}_{i\in S, j\in[N]} \leftarrow \{\mathsf{pk}_i\}_{i\in S}$.
Set $r \leftarrow h_\ell(k)$.
Set $q \leftarrow H_{\mathsf{prime}}(\mathsf{pk}\|m)$.
Set the signer's selected partial secret isogeny $\psi_r \leftarrow \mathsf{sk}_k[\ell] = \varphi_{\ell,r}$, and denote its committed kernel scalar by $\mu_r = \mu_{\ell,r}$.
if $r = 1$ **then**
  if $\Sigma_{\mathsf{prev}} \neq \perp$ **then abort**;
  Compute the deterministic initial signature $\sigma_0 \leftarrow \mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$.
  if $\deg(\sigma_0) \neq q(2^a - q)$ **then abort**;
  Compute $\sigma_r \leftarrow \mathsf{PushForwardSig}(\sigma_0, \psi_r)$.
else
  Parse $\Sigma_{\mathsf{prev}} = \left(\{\sigma_i\}_{0\le i\le r-1}, k_{\mathsf{prev}}, \{\pi_i^{\mathsf{sig}}\}_{1\le i\le r-1}\right)$.
  if $\{k_{\mathsf{prev}}\} \neq h_\ell^{-1}(r-1) \cap S$ **then abort** ;
  if $\sigma_0 \neq \mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$ **then abort** ;
  for $i \leftarrow 1 \ldots r-1$ **do**
    if $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{weak\text{-}push}}}.\mathsf{Verify}^{H_{\mathsf{nizk}}}((\mathsf{Com}_{\ell,i}, q, \sigma_{i-1}, \sigma_i), \pi_i^{\mathsf{sig}}) \neq 1$ **then abort** ;
  if $\deg(\sigma_{r-1}) \neq q(2^a - q)$ **then abort** ;
  Compute $\sigma_r \leftarrow \mathsf{PushForwardSig}(\sigma_{r-1}, \psi_r)$.
if $\deg(\sigma_r) \neq q(2^a - q)$ **then abort** ;
if $r = t$ **then**
  return σ_r
Set the NIZK statement $x_r^{\mathsf{sig}} \leftarrow (\mathsf{Com}_{\ell,r}, q, \sigma_{r-1}, \sigma_r)$.
Generate the NIZK proof $\pi_r^{\mathsf{sig}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{weak\text{-}push}}}.\mathsf{Prove}^{H_{\mathsf{nizk}}}\left(1^\lambda, x_r^{\mathsf{sig}}, \mu_r\right)$.
return $\Sigma_k = \left(\{\sigma_i\}_{0\le i\le r}, k, \{\pi_i^{\mathsf{sig}}\}_{1\le i\le r}\right)$.

Verification is performed using exactly the same verification algorithm of PRISM-sig [10] as described in Algorithm 7.

The security of SPRITZ mainly depends on the hardness assumption defined in Definition 10 of Appendix E. Under the stated assumptions, SPRITZ is proved secure in Theorem 1.

Algorithm 7: $\mathsf{Verify}(\mathsf{pk}, m, \sigma)$

Input: A message $m \in \{0,1\}^*$, a signature σ and the associated public key pk.
Output: True or False

Parse $(E_1, P_1, Q_1, E_2, P_2, Q_2) \leftarrow \sigma$.
if $j(E_1) \neq j(\mathsf{pk})$ **then return** False;
Compute two-dimensional isogeny Φ such that $\ker \Phi = \langle (P_1, P_2), (Q_1, Q_2) \rangle$.
if Φ *is not splitting* **then return** False;
Compute $q \leftarrow H_{\mathsf{prime}}(\mathsf{pk} || m)$.
Compute $(P', _) \leftarrow \Phi((P_1, 0_{E_2}))$ and $(Q', _) \leftarrow \Phi((Q_1, 0_{E_2}))$.
Compute pairings $w \leftarrow \omega_{2^a}(P_1, Q_1)$ and $w' \leftarrow \omega_{2^a}(P', Q')$.
if $w \neq w'^q$ *and* $w' \neq w^q$ **then return** False ;
return True.

Theorem 1. *Assuming the SPRITZ hardness assumption (Definition 10), the heuristic assumption of Definition 4, collision resistance of H_{prime} and H_{key}, and that the NIZK satisfies Definition 7, SPRITZ satisfies correctness and is quantum secure in the static TS-UF-0 threshold security model (Definition 6), with the NIZK proof-system modeled as a quantum random oracle.*

Proof. The proof can be found in Appendix E. □

Remark 1. The final signature contains no NIZK proofs, since these proofs are used during signing to prevent partial-key recovery attacks against honest parties. Consequently, NIZK proofs contribute neither to the final signature size nor to final signature verification time.

4 Performance

We evaluate our proof-of-concept Rust implementation of SPRITZ for $(n, t) = (7, 5)$ using the combination of Spartan and WHIR. We explain the details in Appendix D.3. The source code and reproduction instructions are available at

`https://anonymous.4open.science/r/threshold_prism_zk-4DDF`.

4.1 Experimental setup

Experiments were conducted under WSL2 on an Intel Core i7-9700 CPU at 3.00 GHz with 32 GB of RAM. We used `rustc 1.96.0`, LLVM 22.1.2, release-mode compilation, and eight Rayon workers for NIZK proof generation. The three profiles use $(\lambda, b) = (128, 162)$, $(192, 243)$, and $(256, 323)$, respectively. Each profile includes one fresh randomized key generation and one sequential chain of five signers. Reported means are taken across these five signers, rather than independent benchmark repetitions.

4.2 Running time

Table 2 reports the measured running times. Key generation and signing are more expensive than final signature verification, which takes between 0.072 and 0.511 seconds. The final signature does not contain paritial signatures and NIZK proofs, and the verification requires no NIZK verification. The signature size and public key size are calculated in Appendix C.5 and also shown in Table 1.

Table 2: Timings in seconds for SPRITZ with $(n, t) = (7, 5)$. T_{KeyGen} is the fresh key-generation time. $\overline{T}_{\text{Sign}}$ is the mean per-signer signing time, excluding verification of preceding parties' proofs. $T_{\text{Sign,total}}$ is the total signing time, including those proof verifications. $\overline{T}_{\text{ProofVerify}}$ is the mean time to verify one party's NIZK independently. T_{Verify} is the final signature verification time.

Level	T_{KeyGen}	$\overline{T}_{\text{Sign}}$	$T_{\text{Sign,total}}$	$\overline{T}_{\text{ProofVerify}}$	T_{Verify}
lvl1	48.645	2.690	19.176	0.574	0.075
lvl3	139.134	8.800	63.261	1.892	0.225
lvl5	299.508	17.578	122.080	3.446	0.517

The benchmark performs sequential verification: signer i verifies all $i - 1$ preceding proofs when its signing begins. Consequently, the sequential chain includes $t(t-1)/2 = 10$ proof verifications. In a distributed system, each party can independently verify proofs as they arrive and cache its own verification results. With broadcast and parallel verification, this pipelining can reduce the proof-verification from $O(t^2)$ to $O(t)$. This optimization is not included in the reported timings.

4.3 R1CS and communication sizes

Each signer uses 15,529, 23,265, and 30,835 R1CS constraints over $\mathbb{F}_p$ at levels I, III, and V, respectively. Following the encoding of [32], a degree-3^b isogeny path requires $10b$ constraints over $\mathbb{F}_p$ and the two paths in our relation therefore contribute $20b$ constraints. The complete circuit additionally includes scalar range checks, commitment verification, scalar-to-kernel computations. The total constraint count per signer is $\Theta(\lambda)$. The cost of the underlying field arithmetic also increases with λ. Table 3 reports the NIZK proof and partial-signature sizes. These communication costs arise during signing and do not represent the size of the final signature.

Table 3: Mean sizes over five partial signatures; $1\,\text{MiB} = 2^{20}$ bytes.

Level	Mean NIZK proof (MiB)	Mean partial signature (MiB)
I	6.117	14.129
III	12.949	30.883
V	16.771	49.069

References

1. Leslie Lamport. The implementation of reliable distributed multiprocess systems. *Computer Networks (1976)*, 2(2):95–114, 1978.
2. Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 307–315, New York, NY, 1990. Springer New York.
3. Yvo Desmedt and Yair Frankel. Shared generation of authenticators and signatures. In *Annual International Cryptology Conference*, pages 457–469. Springer, 1991.
4. Michael K Reiter and Kenneth P Birman. How to securely replicate services. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(3):986–1009, 1994.
5. Christopher A. Kent and Jeffrey C. Mogul. Fragmentation considered harmful. *ACM SIGCOMM Computer Communication Review*, 17(5):390–401, oct 1987.
6. Philippe Golle and Nagendra Modadugu. Authenticating streamed data in the presence of random packet loss. In *NDSS*, volume 1, pages 13–22, 2001.
7. Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.
8. Daniel J Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ signature framework. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 2129–2146, 2019.
9. Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. Sqisign: compact post-quantum signatures from quaternions and isogenies. In *International conference on the theory and application of cryptology and information security*, pages 64–93. Springer, 2020.
10. Andrea Basso, Giacomo Borin, Wouter Castryck, Maria Corte-Real Santos, Riccardo Invernizzi, Antonin Leroux, Luciano Maino, Frederik Vercauteren, and Benjamin Wesolowski. Prism: Simple and compact identification and signatures from large prime degree isogenies. In Tibor Jager and Jiaxin Pan, editors, *Public-Key Cryptography – PKC 2025*, pages 300–332, Cham, 2025. Springer Nature Switzerland.
11. Giacomo Borin. PRISM: Compact threshold signatures from pushforwards of large-degree isogenies, 2026. Joint work with Andrea Basso, Riccardo Invernizzi, Luciano Maino, Robi Pedersen, and Maria Corte-Real Santos (https://csrc.nist.gov/presentations/2026/mpts2026-4a2).
12. Luis Teixeira D'Aguiar Norton Brandao and Rene Peralta. Nist first call for multiparty threshold schemes, 2026-01-20 05:01:00 2026.
13. John Kelsey, Nathalie Lang, and Stefan Lucks. Turning hash-based signatures into distributed signatures and threshold signatures. *IACR Communications in Cryptology*, 2(2), 2025.
14. Giacomo Borin, Sofía Celi, Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Guilhem Niot, Thomas Prest, and Kaoru Takemure. Hermine: An efficient lattice-based frost-like threshold signature. *Cryptology ePrint Archive*, 2026.
15. Marco Baldi, Michele Battagliola, Giacomo Borin, Giovanni Di Crescenzo, Rahmi El Mechri, Alessio Meneghetti, Edoardo Persichetti, Paolo Santini, and Floyd Zweydinger. LEAST: Linear equivalence action threshold signature. NIST Multi-Party Threshold Cryptography (MPTC) Preview Writeup, 2026. Preview Writeup, Version 0.1.

16. Rafaël Del Pino and Guilhem Niot. Finally! a compact lattice-based threshold signature. In *IACR International Conference on Public-Key Cryptography*, pages 169–199. Springer, 2025.
17. Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. Quorus: Efficient, scalable threshold ml-dsa signatures from mpc. *USENIX Security 2026*, 2026.
18. Antonín Dufka, Semjon Kravtvsenko, Peeter Laud, and Nikita Snetkov. Trilithium: Efficient and universally composable distributed ml-dsa signing. *Cryptology ePrint Archive*, 2025.
19. Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 219–248. Springer, 2024.
20. Diego F. Aranha, Giacomo Borin, Sofia Celi, and Guilhem Niot. Optimizing and implementing threshold MAYO. Cryptology ePrint Archive, Paper 2026/710, 2026.
21. Sofia Celi, Daniel Escudero, and Guilhem Niot. Share the mayo: thresholdizing mayo. In *International Conference on Post-Quantum Cryptography*, pages 165–198. Springer, 2025.
22. Marius A. Aardal, Shahla Atapoor, Karim Baghery, Andrea Basso, Xavier Bonnetain, Giacomo Borin, Daniele Cozzo, Pierrick Dartois, Luca De Feo, Max Duparc, Jonathan K. Eriksen, Tako Boris Fouotsa, Arthur Herlédan Le Merdy, Riccardo Invernizzi, Samuel Jaques, Yi-Fu Lai, Dania Lazzarini, Jason T. LeGrow, Chloe Martindale, Luciano Maino, Jonas Meers, Michael Meyer, Sikhar Patranabis, Robi Pedersen, Giacomo Pope, Doreen Riepel, Damien Robert, Ryan Rueger, Sina Schaeffler, André Schrottenloher, and Frederik Vercauteren. PQarrots: Macaw, Kea and Kakapo: Threshold primitives from (isogeny-based) group actions. NIST Multi-Party Threshold Cryptography (MPTC) Preview Writeup, 2026. Preview Writeup, Version 0.3.
23. Yini Lin, Muhammed F. Esgin, Amin Sakzad, Ron Steinfeld, and Markku-Juhani O. Saarinen. Lemur: Scalable post-quantum synchronized multi-signatures. Cryptology ePrint Archive, Paper 2026/1161, 2026.
24. Nikai Jagganath, Muhammed F. Esgin, Ron Steinfeld, Amin Sakzad, Markku-Juhani O. Saarinen, and Dongxi Liu. LoTRS: Practical post-quantum structured threshold ring signatures from lattices. Cryptology ePrint Archive, Paper 2026/974, 2026.
25. Leo Kao and Raymond Chang. TALUS: FIPS-204-exact threshold ML-DSA via boundary clearance, 2026.
26. Andreas Huelsing, Denis Butin, Stefan-Lukas Gazdag, Joost Rijneveld, and Aziz Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, 2018.
27. David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, 2019.
28. Giacomo Borin, Maria Corte-Real Santos, Jonathan Komada Eriksen, Riccardo Invernizzi, Marzio Mula, Sina Schaeffler, and Frederik Vercauteren. Qlapoti: Simple and efficient translation of quaternion ideals to isogenies. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 174–205, Singapore, 2026. Springer Nature Singapore.
29. David Kohel, Kristin Lauter, Christophe Petit, and Jean-Pierre Tignol. On the quaternion ℓ-isogeny path problem. *LMS Journal of Computation and Mathematics*, 17(A):418–432, 2014.

30. Luciano Maino and Christophe Petit. Pushforward problems and applications to isogeny-based cryptography. Cryptology ePrint Archive, Paper 2026/1030, 2026.
31. Andrea Basso and Luciano Maino. Poké: A compact and efficient pke from higher-dimensional isogenies. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 94–123, Cham, 2025. Springer Nature Switzerland.
32. Yi-Fu Lai and Luciano Maino. Toward zksnark-assisted isogeny-based cryptography. *Cryptology ePrint Archive*, 2026.
33. Joseph H Silverman. *The arithmetic of elliptic curves*, volume 106. Springer, 2009.
34. Luca De Feo. Mathematics of isogeny based cryptography. *arXiv preprint arXiv:1711.04062*, 2017.
35. Jacques Vélu. Isogénies entre courbes elliptiques. *Comptes-Rendus de l'Académie des Sciences*, 273:238–241, 1971.
36. John Voight. *Quaternion algebras*. Springer, 2021.
37. Antonin Leroux. Quaternion algebra and isogeny-based cryptography. *Theses, Ecole doctorale de l'Institut Polytechnique de Paris*, 2022.
38. Max Deuring. Die typen der multiplikatorenringe elliptischer funktionenkörper. In *Abhandlungen aus dem mathematischen Seminar der Universität Hamburg*, volume 14, pages 197–272. Springer Berlin/Heidelberg, 1941.
39. Tomoyoshi Ibukiyama. On maximal orders of division quaternion algebras over the rational number field with certain optimal embeddings. *Nagoya Mathematical Journal*, 88:181–195, 1982.
40. Ernst Kani. The number of curves of genus two with elliptic differentials. *Journal für die reine und angewandte Mathematik*, 485:93–122, 1997.
41. Andrea Basso, Luciano Maino, and Giacomo Pope. Festa: Fast encryption from supersingular torsion attacks. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 98–126, Singapore, 2023. Springer Nature Singapore.
42. Kohei Nakagawa and Hiroshi Onuki. Qfesta: Efficient algorithms and parameters for festa using quaternion algebras. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 75–106, Cham, 2024. Springer Nature Switzerland.
43. Mihir Bellare, Elizabeth Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 517–550, Cham, 2022. Springer Nature Switzerland.
44. Kurt Mehlhorn. *Data structures and algorithms 1: Sorting and searching*. Springer-Verlag, 1984.
45. M Atici, Douglas R Stinson, and R Wei. A new practical algorithm for the construction of a perfect hash function. *Journal of Combinatorial Mathematics and Combinatorial Computing*, 35:127–146, 2000.
46. Douglas R Stinson, Ruizhong Wei, and L Zhu. New constructions for perfect hash families and related structures using combinatorial designs and codes. *Journal of Combinatorial Designs*, 8(3):189–200, 2000.
47. Moni Naor, Leonard J Schulman, and Aravind Srinivasan. Splitters and near-optimal derandomization. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*, pages 182–191. IEEE, 1995.
48. Luca De Feo, Antonin Leroux, Patrick Longa, and Benjamin Wesolowski. New algorithms for the deuring correspondence. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 659–690, Cham, 2023. Springer Nature Switzerland.

49. Maria Corte-Real Santos, Craig Costello, and Benjamin Smith. Efficient (3, 3)-isogenies on fast kummer surfaces. *Research in Number Theory*, 11(1):25, 2025.
50. G.H. Hardy and J.E. Littlewood. Some problems of'partitio numerorum'; iii: On the expression of a number as a sum of primes. *Goldbach Conjecture*, 4:21, 1984.
51. Jorge Chavez-Saab, Maria Corte-Real Santos, Luca De Feo, Jonathan Komada Eriksen, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Michael Meyer, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Francisco Rodríguez-Henríquez, Sina Schaeffer, and Benjamin Wesolowski. Sqisign specification. `https://sqisign.org/spec/sqisign-20250707.pdf`, 2025. Accessed: 2026-04-26.
52. Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the poseidon hash function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023*, pages 177–203, Cham, 2023. Springer Nature Switzerland.
53. Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 704–737, Cham, 2020. Springer International Publishing.
54. Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. Whir: Reed–solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 214–243, Cham, 2025. Springer Nature Switzerland.
55. Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed–solomon codes. *Journal of the ACM*, 70(5):1–57, 2023.
56. Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On proximity gaps for Reed–Solomon codes. Cryptology ePrint Archive, Paper 2025/2055, 2025.
57. Dmitry Krachun, Stepan Kazanin, and Ulrich Haböck. Failure of proximity gaps close to capacity. *Cryptology ePrint Archive*, 2026.
58. Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. Cryptology ePrint Archive, Paper 2026/391, 2026.
59. Alessandro Chiesa, Peter Manohar, and Nicholas Spooner. Succinct arguments in the quantum random oracle model. In Dennis Hofheinz and Alon Rosen, editors, *Theory of Cryptography*, pages 1–29, Cham, 2019. Springer International Publishing.
60. Alessandro Chiesa, Zijing Di, Zihan Hu, and Yuxi Zheng. How to prove post-quantum security for succinct non-interactive reductions. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology – EUROCRYPT 2026*, pages 389–415, Cham, 2026. Springer Nature Switzerland.
61. Jonathan Komada Eriksen, Lorenz Panny, Jana Sotáková, and Mattia Veroni. Deuring for the people: supersingular elliptic curves with prescribed endomorphism ring in general characteristic. *LuCaNT: LMFDB, computation, and number theory*, 796:339–373, 2023.
62. Jung Hee Cheon, Wonhee Cho, and Jiseung Kim. Improved universal thresholdizer from iterative shamir secret sharing. *Journal of Cryptology*, 38(1), January 2025.

A Detailed preliminaries

A.1 Isogenies and quaternion algebra

In this subsection, we briefly introduce the basic notions about isogenies used in this paper. For more details, see [33,34].

Throughout this paper, every elliptic curve is supersingular and defined over $\mathbb{F}_{p^2}$, unless otherwise stated. An *isogeny* $\phi : E \to E'$ between elliptic curves E and E' is a nonconstant morphism preserving the identity element. We write $\mathrm{Hom}(E, E')$ for the group of homomorphisms between E and E', including the zero homomorphism, and $\mathrm{End}(E) := \mathrm{Hom}(E, E)$ for the endomorphism ring of E where the multiplication is endowed by the composition. For an isogeny ϕ, we denote its image curve by $\mathrm{Im}(\phi)$.

The *degree* of an isogeny ϕ is its degree as a finite morphism, denoted by $\deg(\phi)$. We say that ϕ is a d-isogeny when $\deg(\phi) = d$. If $\psi : E' \to E''$ is another isogeny, then

$$\deg(\psi \circ \phi) = \deg(\psi)\deg(\phi).$$

An isogeny ϕ is *separable* if the induced extension of function fields is separable. For separable isogenies, we have $\#\ker(\phi) = \deg(\phi)$. In particular, every isogeny of degree coprime to p is separable.

To avoid confusion with the notation $[n] := \{1, \ldots, n\}$, we write $[m]_E$ for the multiplication-by-m map on E, and we denote the m-torsion subgroup by

$$E[m] := \ker([m]_E).$$

If $\gcd(m, p) = 1$, then $E[m] \cong (\mathbb{Z}/m\mathbb{Z})^2$. More generally, for every finite subgroup $G \subseteq E$ there exists a quotient isogeny

$$\phi_G : E \to E/G,$$

unique up to isomorphism of the codomain, with kernel G [35]. A separable isogeny is called *cyclic* if its kernel is cyclic. Hence a cyclic m-isogeny ϕ may be represented by a single point P such that $\langle P \rangle = \ker(\phi)$. This is the kernel representation of a cyclic isogeny. Whenever we use a kernel representation, we implicitly restrict to cyclic isogenies unless otherwise stated.

For every nonzero isogeny $\phi : E \to E'$ there exists a unique *dual isogeny*

$$\widehat{\phi} : E' \to E$$

such that

$$\widehat{\phi} \circ \phi = [\deg(\phi)]_E, \qquad \phi \circ \widehat{\phi} = [\deg(\phi)]_{E'}.$$

Finally, if $\phi : E \to E_1$ and $\psi : E \to E_2$ are separable isogenies with $\gcd(\deg(\phi), \deg(\psi)) = 1$, then $\phi(\ker\psi) \subseteq E_1$ is a finite subgroup, and the quotient isogeny

$$[\phi]_*\psi : E_1 \to E_1/\phi(\ker\psi)$$

is called the *push-forward* of ψ along ϕ. Given $\phi : E_1 \to E$ and $\psi : E \to E_2$, the *pull-back* of ψ along ϕ is the push-forward of ψ along the dual isogeny $\widehat{\phi}$, which is denoted by $[\phi]^*\psi = [\widehat{\phi}]_*\psi : E_1 \to E_1/\hat{\phi}(\ker\psi)$.

For an elliptic curve E over $\mathbb{F}_{p^2}$ and an integer k coprime to p, we denote by $\omega_{E,k} : E[k] \times E[k] \longrightarrow \mu_k$ the k-Weil pairing, where $\mu_k \subset \overline{\mathbb{F}}_p^{\times}$ is the group of k-th roots of unity. This pairing is bilinear, alternating, and non-degenerate. When the curve E is clear from context, we simply write ω_k.

By the same convention, a homomorphism between products of n elliptic curves is also called an *n-dimensional isogeny* if it is surjective with finite kernel.

To simplify computations in isogeny-based cryptography, we use quaternion algebras related to the supersingular elliptic curves defined over $\mathbb{F}_{p^2}$. For more details of quaternion algebras, see [36,37].

Quaternion algebras Let $B_{p,\infty}$ denote a quaternion algebra over $\mathbb{Q}$ whose $\mathbb{Q}$-basis is $\{1, i, j, k := ij\}$ with $i^2 = -1, j^2 = -p, ij = -ji$. Every quaternion element can be represented by $\alpha = a + bi + cj + dk \in B_{p,\infty}$ for some $a, b, c, d \in \mathbb{Q}$. The canonical conjugate $\bar{\alpha}$ is defined as $a - bi - cj - dk$. The reduced trace and reduced norm are defined by

$$\mathrm{trd}(\alpha) := \alpha + \bar{\alpha},\ \mathrm{nrd}(\alpha) := \alpha\bar{\alpha} = \bar{\alpha}\alpha \in \mathbb{Q}.$$

A *fractional ideal* in $B_{p,\infty}$ is a $\mathbb{Z}$-submodule of rank 4. An *order* $\mathcal{O}$ is a fractional ideal that is also a subring of $B_{p,\infty}$. An order $\mathcal{O}$ is *maximal* if it is not properly contained in any other order. A fractional ideal I is called a left $\mathcal{O}$-ideal if $\alpha I \subseteq I$ for all $\alpha \in \mathcal{O}$. It is called a right $\mathcal{O}$-ideal if $I\alpha \subseteq I$ for all $\alpha \in \mathcal{O}$. For a fractional ideal I, its left and right orders are defined by

$$O_L(I) := \{\alpha \in B_{p,\infty} : \alpha I \subseteq I\}, \qquad O_R(I) := \{\alpha \in B_{p,\infty} : I\alpha \subseteq I\}.$$

Its conjugate is defined by $\bar{I} := \{\bar{\alpha} : \alpha \in I\}$ and its reduced norm is defined by $\mathrm{nrd}(I) := \gcd\{\mathrm{nrd}(\alpha) : \alpha \in I\}$. For a fractional ideal I whose left and right orders are maximal, we define its inverse fractional ideal by $I^{-1} := \frac{\bar{I}}{\mathrm{nrd}(I)}$. We say that I is integral if it is contained in its left order (or, equivalently, to its right order). Moreover, I is said to be cyclic if for all prime ℓ we have that $I \not\subset \ell O_L(I)$. Two left $\mathcal{O}$-ideals I and J are said to be *equivalent*, written $I \sim J$, if there exists $\beta \in B_{p,\infty}^{\times}$ such that $J = I\beta$. In this case,

$$O_L(J) = O_L(I) \qquad \text{and} \qquad O_R(J) = \beta^{-1}O_R(I)\beta.$$

The corresponding equivalence classes form the *left ideal class set* of $\mathcal{O}$, which is the object that appears on the quaternionic side of the Deuring correspondence.

Deuring correspondence. For a supersingular elliptic curve E defined over $\mathbb{F}_{p^2}$, the endomorphism ring $\mathrm{End}(E)$ is isomorphic to a maximal order in $B_{p,\infty}$ [38]. Conversely, every maximal order in $B_{p,\infty}$ occurs as $\mathrm{End}(E)$ for some supersingular elliptic curve E. Thus, on objects, the Deuring correspondence identifies

isomorphism classes of supersingular elliptic curves with isomorphism classes of maximal orders in $B_{p,\infty}$.

For a separable isogeny $\varphi : E \to E'$, we define the associated connecting ideal by

$$I_\varphi := \{\alpha \circ \varphi : \alpha \in \mathrm{Hom}(E', E)\}.$$

Then

$$\mathrm{nrd}(I_\varphi) = \deg(\varphi).$$

Equivalent isogenies give equivalent ideals, the dual isogeny corresponds to the conjugate ideal,

$$I_{\widehat{\varphi}} = \overline{I_\varphi},$$

and for composable isogenies $\varphi : E \to E'$ and $\psi : E' \to E''$,

$$I_{\psi\circ\varphi} = I_\varphi I_\psi.$$

We also define the pull-back of ideals I, J such that $\mathcal{O}_R(I) = \mathcal{O}_L(J)$ and $\gcd(\mathrm{nrd}(I), \mathrm{nrd}(J)) = 1$ as

$$[I]^* J := IJ + \mathrm{nrd}(J)\mathcal{O}_L(I).$$

For an integral left $\mathcal{O}_0$-ideal I whose reduced norm is coprime to p, define

$$E_0[I] := \{P \in E_0(\overline{\mathbb{F}}_p) : \rho_0(\alpha)(P) = 0_{E_0} \text{ for every } \alpha \in I\}.$$

Under the Deuring correspondence, $E_0[I]$ is the kernel of the separable isogeny

$$\phi_I : E_0 \longrightarrow E_0/E_0[I]$$

associated with I.

Choose $\sqrt{-1} \in \mathbb{F}_{p^2}$ such that $\sqrt{-1}^2 = -1$, and define endomorphisms in $\mathrm{End}(E_0)$

$$\iota(x, y) := (-x, \sqrt{-1}y), \qquad \pi(x, y) := (x^p, y^p).$$

Then $\iota^2 = [-1]$, $\pi^2 = [-p]$, and $\iota\pi = -\pi\iota$. The map

$$\rho_0 : \ B_{p,\infty} \longrightarrow \mathrm{End}(E_0) \otimes_{\mathbb{Z}} \mathbb{Q}, \qquad i \mapsto \iota, \quad j \mapsto \pi, \quad k = ij \mapsto \iota\pi,$$

restricts to an isomorphism [39]

$$\rho_0|_{\mathcal{O}_0} : \ \mathcal{O}_0 := \mathbb{Z}\Big\langle 1,\, i,\, \frac{i+j}{2},\, \frac{1+k}{2} \Big\rangle \xrightarrow{\sim} \mathrm{End}(E_0).$$

We give a table which summarizes the Deuring correspondence, the correspondence between objects of elliptic curves and objects of quaternion algebras.

Elliptic-curve world	**Quaternion-algebra world**
Supersingular elliptic curve E	$\mathcal{O} = \mathrm{End}(E) \subset B_{p,\infty}$
$\varphi : E \to E'$	I_φ, a $\mathcal{O}$,$\mathcal{O}' = \mathrm{End}(E')$-connecting ideal
$\deg(\varphi)$	$\mathrm{nrd}(I_\varphi)$
$\varphi, \psi : E \to E'$	$I_\varphi \sim I_\psi$
$\widehat{\varphi} : E' \to E$	$I_{\widehat{\varphi}} = \overline{I_\varphi}$
$\psi \circ \varphi$	$I_\varphi \cdot I_\psi$
$[\varphi]^*\psi$	$[I_\varphi]^* I_\psi$

Kani's Lemma Now we recall Kani's lemma that uses two-dimensional representation arising from an isogeny commutative diagram.

Theorem 2 (Kani's Lemma [40]). *Consider the following commutative diagram of isogenies*

$$\begin{array}{ccc} E_0 & \xrightarrow{\phi_1} & E_1 \\ {\scriptstyle \phi_2}\downarrow & & \downarrow{\scriptstyle \phi_2'} \\ E_2 & \xrightarrow[\phi_1']{} & E_{12} \end{array}$$

where $\deg(\phi_i) = \deg(\phi_i') = d_i$ *and* $\gcd(d_1, d_2) = 1$. *Then, the map*

$$\Phi = \begin{pmatrix} \phi_1 & -\hat{\phi}_2' \\ \phi_2 & \hat{\phi}_1' \end{pmatrix} : E_0 \times E_{12} \to E_1 \times E_2$$

is a 2*-dimensional isogeny whose kernel is given by*

$$\mathrm{Ker}\Phi = \{([-d_1]_{E_0} P, \phi_2' \circ \phi_1(P)) | P \in E_0[d_1 + d_2]\}.$$

Kani's lemma enables the following *2-dimensional representation* for isogenies of degree $q(2^a - q)$, which is used in FESTA [41] and QFESTA [42]. Let E be an elliptic curve such that $2^a | \#E(\mathbb{F}_{p^2})$. An isogeny $\phi : E \to E'$ admits an efficient representation

$$(q, E, E', (P_0, Q_0), (\phi(P_0), \phi(Q_0)))\,,$$

where (P_0, Q_0) is a basis of $E[2^a]$ and $\deg(\phi) = q(2^a - q)$ for odd $q < 2^a$.

The 2-dimensional representation also allows us to build an efficient algorithm IdealToIsogeny, which converts an input $\mathcal{O}_0$-ideal into an output corresponding isogeny ϕ such that $\mathrm{dom}(\phi) = E_0$. To build such algorithm, we use the subroutine algorithm Qlapoti [28], which computes two output equivalent ideals I_1, I_2 of an input left $\mathcal{O}_0$-ideal I such that $\mathrm{nrd}(I_1) + \mathrm{nrd}(I_2) = 2^e$ where $2^e \mid (p + 1)$. [3] For more details, see [28].

[3] Note that we slightly modify the original Qlapoti algorithm to get output ideals I_1, I_2 satisfying $\mathrm{nrd}(I_1) + \mathrm{nrd}(I_2) = 2^a 3^b$ where a, b are parameters of our constructions. For more details, see Section C.3.

A.2 Properties of threshold signature scheme

Definition 5 (Correctness). *For any fixed integers* $1 \leq t \leq n$*, let*

$$\mathsf{TSS} = \big(\mathsf{Setup}, \mathsf{KeyGen}, (\mathsf{Sign}_r)_{r\in[R]}, \mathsf{Combine}, \mathsf{Verify}\big)$$

be an R*-round* (n,t)*-threshold signature scheme. We say that* TSS *is* correct *if there exists a negligible function* ϵ *such that, for every* $\lambda \in \mathbb{N}$*, message* msg*, and* $S \in \Gamma_{n,t}$*,*

$$\Pr\left[\begin{array}{l} \mathsf{pp} \xleftarrow{\$} \mathsf{Setup}(\lambda, n, t);\ (\mathsf{pk}, (\mathsf{sk}_i)_{i\in[n]}, (\mathsf{pk}_i)_{i\in[n]}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{pp}); \\ \mathsf{pm}_0^i \leftarrow \perp \textit{ for all } i \in S; \\ \mathsf{pm}_r^i \xleftarrow{\$} \mathsf{Sign}_r\big(\mathsf{pk}, S, (\mathsf{pk}_j)_{j\in S}, \mathsf{msg}, (\mathsf{pm}_{r-1}^j)_{j\in S}, i, \mathsf{sk}_i\big) \\ \textit{for all } r \in [R],\ i \in S; \\ \mathsf{sig} \leftarrow \mathsf{Combine}\big(\mathsf{pk}, S, \mathsf{msg}, (\mathsf{pm}_r^i)_{r\in[R],\, i\in S}\big) : \\ \Big(\bigwedge_{r\in[R],\, i\in S} \mathsf{pm}_r^i \neq \perp\Big) \wedge \mathsf{Verify}(\mathsf{pk}, \mathsf{msg}, \mathsf{sig}) = 1 \end{array}\right] \geq 1 - \epsilon(\lambda).$$

Security. We formalize security against a static malicious adversary using the standard TS-UF-0 rule [43]. In TS-UF-0, a signature of the message m generated by the adversary is taken as trivial if a partial signature of m was generated by at least one honest party. Here, "static" means only that the corruption set is fixed before key generation. During signing, the adversary controls the corrupted parties and the message delivery, and it may query an individual honest party on an adversarially chosen incoming transcript.

Definition 6 (Static threshold EUF-CMA security (TS-UF-0)). *Let*

$$\mathsf{TSS} = \big(\mathsf{Setup}, \mathsf{KeyGen}, (\mathsf{Sign}_r)_{r\in[R]}, \mathsf{Combine}, \mathsf{Verify}\big)$$

be an R*-round* (n,t)*-threshold signature scheme as in Definition 1. For a quantum polynomial-time (QPT) adversary* $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$*, define the following experiment:*

Experiments for static threshold EUF-CMA security

$\underline{\mathsf{ExpPhase1}^{\mathsf{st\text{-}TSS\text{-}EUF\text{-}CMA}}_{\mathsf{Setup},\mathcal{A}_0}(\lambda, n, t)}$

1. $\mathsf{pp} \leftarrow \mathsf{Setup}(\lambda, n, t)$.
2. $(C, \mathsf{aux}) \leftarrow \mathcal{A}_0(\mathsf{pp}, n, t, R)$.
   If $C \not\subseteq [n]$ *or* $|C| \geq t$, *output* 0.
3. *Output* $(C, \mathsf{aux}, \mathsf{pp})$.

$\underline{\mathsf{ExpPhase2}^{\mathsf{st\text{-}TSS\text{-}EUF\text{-}CMA}}_{\mathsf{TSS},\mathcal{A}_1}(n, t, C, \mathsf{aux}, \mathsf{pp})}$

1. $\big(\mathsf{pk}, (\mathsf{sk}_i)_{i\in[n]}, (\mathsf{pk}_i)_{i\in[n]}\big) \leftarrow \mathsf{KeyGen}(\mathsf{pp})$.
2. $Q \leftarrow \emptyset$.
3. $(\mathsf{msg}^\star, \mathsf{sig}^\star) \leftarrow \mathcal{A}_1^{\mathsf{Orc}_{\mathsf{part}}}\big(\mathsf{aux}, \mathsf{pp}, \mathsf{pk}, (\mathsf{pk}_i)_{i\in[n]}, (\mathsf{sk}_i)_{i\in C}\big)$.
4. *Output* 1 *if* $\mathsf{Verify}(\mathsf{pk}, \mathsf{msg}^\star, \mathsf{sig}^\star) = 1$ *and* $\mathsf{msg}^\star \notin Q$.
   Output 0 *otherwise.*

The full experiment first runs Phase 1. If Phase 1 outputs 0*, the full experiment outputs* 0*. Otherwise, it passes* $(C, \mathsf{aux}, \mathsf{pp})$ *to Phase 2 and returns the Phase 2 output.*

The state aux *may be quantum. Partial-signing-oracle queries and responses are classical.*

The partial-signing oracle $\mathsf{Orc}_{\mathsf{part}}$ *is defined as follows. Its input* $\Pi_{r-1} = (\widetilde{\mathsf{pm}}^j_{r-1})_{j\in S}$ *is the incoming transcript supplied to party* i *in round* r*.*

$\underline{\mathsf{Orc}_{\mathsf{part}}(r, i, S, \mathsf{msg}, \Pi_{r-1})}$

1. *If* $r \notin [R]$, $S \not\subseteq [n]$, $|S| < t$, *or* $i \notin S \setminus C$, *return* $\perp$.
2. *If* $r = 1$ *and* $\Pi_0 \neq (\perp)_{j\in S}$, *return* $\perp$.
3. $\mathsf{pm}^i_r \leftarrow \mathsf{Sign}_r\big(\mathsf{pk}, S, (\mathsf{pk}_j)_{j\in S}, \mathsf{msg}, \Pi_{r-1}, i, \mathsf{sk}_i\big)$.
4. *If* $\mathsf{pm}^i_r \neq \perp$, *set* $Q \leftarrow Q \cup \{\mathsf{msg}\}$.
5. *Return* pm^i_r.

For $r > 1$*, the adversary chooses the entire incoming transcript* Π_{r-1}*. The challenger does not complete the other parties' steps. Any transcript validation is performed by* Sign_r *itself. The adversary computes messages of corrupted parties locally from* $(\mathsf{sk}_i)_{i\in C}$ *and may freely repeat or interleave partial-signing queries. The algorithms* Combine *and* Verify *are public and may also be run locally.*

The set Q *implements TS-UF-0 freshness. A message enters* Q *as soon as one honest party returns a partial-signing response on that message.*

The advantage of $\mathcal{A}$ *is defined as*

$$\mathsf{Adv}^{\mathsf{st\text{-}TSS\text{-}EUF\text{-}CMA}}_{\mathsf{TSS},\mathcal{A}}(\lambda, n, t) = \Pr\left[\mathsf{Exp}^{\mathsf{st\text{-}TSS\text{-}EUF\text{-}CMA}}_{\mathsf{TSS},\mathcal{A}}(\lambda, n, t) \neq 0\right].$$

We say that TSS *is* statically threshold EUF-CMA secure *if, for every QPT adversary* $\mathcal{A}$*,*

$$\mathsf{Adv}^{\mathsf{st\text{-}TSS\text{-}EUF\text{-}CMA}}_{\mathsf{TSS},\mathcal{A}}(\lambda, n, t) \leq \mathsf{negl}(\lambda).$$

A.3 Properties of non-interactive zero-knowledge proofs

Definition 7 (Adaptive multi-theorem NIZK argument of knowledge in the QROM). *The system* Π *is an* adaptive multi-theorem computational non-interactive zero-knowledge argument of knowledge *for* R_λ *if the following properties hold.*

1. ***Completeness.*** *There exists a negligible function* $\mathsf{negl}_{\mathsf{c}}$ *such that, for every* λ *and every* (x, w) *satisfying* $R_\lambda(x, w) = 1$*,*

$$\Pr_{\substack{H \leftarrow \mathsf{RO}_\lambda \\ \pi \leftarrow \mathsf{Prove}^H(1^\lambda, x, w)}} \left[\mathsf{Verify}^H(x, \pi) = 1\right] \geq 1 - \mathsf{negl}_{\mathsf{c}}(\lambda),$$

   where $H \leftarrow \mathsf{RO}_\lambda$ *denotes sampling the random oracle.*

2. ***Adaptive knowledge soundness.*** *For every QPT prover* $\mathcal{P}^*$*, there exists an expected-QPT extractor* $\mathsf{Ext}^{\mathcal{P}^*}$ *such that the following holds for every polynomial-size auxiliary quantum state independent of the initial random oracle.*

   The extractor executes $\mathcal{P}^*$ *while internally emulating its quantum access to the random oracle. Let* (x, π) *be the prover's output in this execution, let* w *be the extracted witness, and let* $b = \mathsf{Verify}^{\widehat{H}}(x, \pi)$*, where* $\widehat{H}$ *denotes the emulated oracle. The joint distribution of* (x, π, b) *and any classical auxiliary output of the prover is statistically indistinguishable from that of the corresponding real execution with* H*. Moreover,*

$$\Pr[b = 1 \ \wedge \ R_\lambda(x, w) = 0] \leq \mathsf{negl}_{\mathsf{ks}}(\lambda),$$

   where $R_\lambda(x, \perp) = 0$*. The statement* x *may be chosen adaptively after polynomially many oracle queries and preceding protocol messages.*

3. ***Adaptive multi-theorem zero knowledge.*** *There exists a stateful online QPT simulator* $\mathcal{S}$ *such that, for every QPT distinguisher* $\mathcal{D}$ *with polynomial-size auxiliary quantum input independent of the initial random oracle,*

$$\Big|\Pr[\mathsf{Real}_{\Pi,\mathcal{D}}(1^\lambda) = 1] - \Pr[\mathsf{Sim}_{\mathcal{S},\mathcal{D}}(1^\lambda) = 1]\Big| \leq \mathsf{negl}(\lambda).$$

   In both experiments, $\mathcal{D}$ *has quantum access to the proof-system oracle and may make polynomially many adaptive classical proof requests* (x, w)*. Requests with* $R_\lambda(x, w) \neq 1$ *return* $\perp$*.*

   In Real*, valid requests are answered by* $\mathsf{Prove}^H(1^\lambda, x, w)$*. In* Sim*, each valid request is answered by* $\mathcal{S}$ *given only* x*. The simulator may program the oracle*

H at polynomially many classical inputs and the distinguisher accesses this oracle through quantum superpositions. The simulator retains its state across requests. Both experiments output the distinguisher's final bit.

Remark 2 (Meaning of adaptive multi-theorem). "Multi-theorem" means that polynomially many proofs use the same random oracle, and "adaptive" means that each true statement may depend on prior oracle interactions and all earlier proofs. The simulator is online because it answers the current request before seeing the next adaptive choice. Adaptive multi-theorem zero knowledge says that honest proof components reveal no witness information beyond their public statements.

A.4 PRISM signature

PRISM is a hash-and-sign signature scheme based on large prime degree supersingular isogenies [10].

Setting. Fix a prime

$$p = 2^a \cdot f - 1 \equiv 3 \pmod 4$$

for a small odd integer $f > 0$, and an integer $a \approx 2\lambda$ for the security parameter λ, such that the supersingular curves used by the scheme have $\mathbb{F}_{p^2}$-rational 2^a-torsion. Let

$$\mathsf{Primes}_a := \{\, q \text{ prime} : 2^{a-1} < q < 2^a \,\},$$

and fix a basis (P_0, Q_0) of $E_0[2^a]$. For $q \in \mathsf{Primes}_a$, the auxiliary factor $2^a - q$ is used so that a $q(2^a - q)$-isogeny admits a two-dimensional representation via Kani's lemma.

Hard problems. We use the following two computational assumptions.

- *Supersingular endomorphism ring problem:* given a supersingular elliptic curve $E/\mathbb{F}_{p^2}$, compute generators of $\mathrm{End}(E)$.
- *Special-degree isogeny problem:* given a supersingular curve $E/\mathbb{F}_{p^2}$, a prime $q \in \mathsf{Primes}_a$ never queried to SPEDIO_E, and an oracle SPEDIO_E which takes as input a prime $q' \in \mathrm{Primes}_a$ then returns a uniformly random cyclic isogeny of degree $q'(2^a - q')$ from E, compute a cyclic isogeny $\psi : E \to E'$ of degree $q(2^a - q)$ without knowledge of $\mathrm{End}(E)$.

Key recovery is essentially the first problem, whereas unforgeability is tied to the second problem and the collision resistance of H_{prime}.

Subroutines. We use the following auxiliary algorithms.

- $\mathsf{RandomFixedNormIdeal}(\mathcal{O}_0, N)$: on input the maximal order $\mathcal{O}_0$ and a positive integer N, outputs a uniformly random cyclic left $\mathcal{O}_0$-ideal of reduced norm N.

– $\mathsf{IdealToIsogeny}(I)$: on input a left $\mathcal{O}_0$-ideal I, outputs the corresponding isogeny $\phi_I : E_0 \to E_I$ together with E_I, $\phi_I(P_0)$, and $\phi_I(Q_0)$.
– $\mathsf{H}_{\mathrm{prime}} : \{0,1\}^* \to \mathrm{Primes}_a$: a hash-to-primes map. Concretely, given a hash $H_{a-2} : \{0,1\}^* \to [0, 2^{a-2})$, define $\mathsf{H}_{\mathrm{prime}}(X)$ as the first prime among

$$2^{a-1} + 2H_{a-2}(X\|c) + 1, \qquad c = 0, 1, 2, \ldots.$$

A degree-$q(2^a - q)$ isogeny $\psi : E \to E'$ is represented by

$$(E', \widetilde{P}, \widetilde{Q}), \qquad \widetilde{P} = [q^{-1}]\psi(P_E), \qquad \widetilde{Q} = [q^{-1}]\psi(Q_E),$$

where (P_E, Q_E) is a deterministic basis of $E[2^a]$ and $q^{-1} \in (\mathbb{Z}/2^a\mathbb{Z})^\times$.

Key Generation. The key generation algorithm of PRISM is as follows:

$\mathsf{KeyGen}(p)$

1. Sample a random left $\mathcal{O}_0$-ideal I_{sk} of norm $N_{\mathsf{sk}} = \ell_{\mathrm{large}}^n$, where $\ell_{\mathrm{large}} > 2^a$ is prime and $n \xleftarrow{\$} \mathbb{N}$ uniformly.
2. Compute the corresponding isogeny $\phi_{\mathsf{sk}} : E_0 \to E_{\mathsf{pk}}$ using $\mathsf{IdealToIsogeny}(I_{\mathsf{sk}})$.
3. Deterministically compute a basis $(P_{\mathsf{pk}}, Q_{\mathsf{pk}})$ of $E_{\mathsf{pk}}[2^a]$.
4. Compute coefficients $m_{11}, m_{12}, m_{21}, m_{22} \in \mathbb{Z}/2^a\mathbb{Z}$ such that

$$P_{\mathsf{pk}} = [m_{11}]\phi_{\mathsf{sk}}(P_0) + [m_{12}]\phi_{\mathsf{sk}}(Q_0), \quad Q_{\mathsf{pk}} = [m_{21}]\phi_{\mathsf{sk}}(P_0) + [m_{22}]\phi_{\mathsf{sk}}(Q_0).$$

5. Output $\mathsf{pk} = (E_{\mathsf{pk}}, P_{\mathsf{pk}}, Q_{\mathsf{pk}})$ and $\mathsf{sk} = (\mathsf{pk}, I_{\mathsf{sk}}, \phi_{\mathsf{sk}}, m_{11}, m_{12}, m_{21}, m_{22})$.

Signing. The signing algorithm of PRISM is as follows:

$\mathsf{Sign}(\mathsf{pk}, \mathsf{sk}, m)$

1. Parse

$$sk = (\mathsf{pk}, I_{\mathsf{sk}}, \phi_{\mathsf{sk}}, m_{11}, m_{12}, m_{21}, m_{22}), \qquad \mathsf{pk} = (E_{\mathsf{pk}}, P_{\mathsf{pk}}, Q_{\mathsf{pk}}).$$

2. Compute

$$q \leftarrow \mathsf{H}_{\mathrm{prime}}(E_{\mathsf{pk}}\|m), \qquad N \leftarrow q(2^a - q).$$

3. Sample

$$I_{\mathsf{ch}} \leftarrow \mathsf{RandomFixedNormIdeal}(\mathcal{O}_0, N),$$

set

$$I \leftarrow I_{\mathsf{sk}} \cap I_{\mathsf{ch}},$$

and compute

$$\phi \leftarrow \mathsf{IdealToIsogeny}(I).$$

4. Let E_{sig} be the codomain of ϕ, and compute
$$P_{\mathsf{sig}} = [m_{11}]\phi(P_0) + [m_{12}]\phi(Q_0),\ Q_{\mathsf{sig}} = [m_{21}]\phi(P_0) + [m_{22}]\phi(Q_0).$$
5. Compute
$$\widetilde{P}_{\mathsf{sig}} = [q^{-1}]P_{\mathsf{sig}}, \qquad \widetilde{Q}_{\mathsf{sig}} = [q^{-1}]Q_{\mathsf{sig}},$$
where $q^{-1} \in (\mathbb{Z}/2^a\mathbb{Z})^\times$.
6. Output
$$\sigma = (E_{\mathsf{sig}}, \widetilde{P}_{\mathsf{sig}}, \widetilde{Q}_{\mathsf{sig}}).$$

Verification. The verification algorithm of PRISM is as follows:

$\underline{\mathsf{Verify}(\mathsf{pk}, \sigma, m)}$
1. Parse
$$\mathsf{pk} = (E_{\mathsf{pk}}, P_{\mathsf{pk}}, Q_{\mathsf{pk}}), \qquad \sigma = (E_{\mathsf{sig}}, \widetilde{P}_{\mathsf{sig}}, \widetilde{Q}_{\mathsf{sig}}).$$
2. Compute
$$q \leftarrow \mathsf{H}_{\mathrm{prime}}(E_{\mathsf{pk}} \| m).$$
3. Set
$$K := \left\langle (P_{\mathsf{pk}}, \widetilde{P}_{\mathsf{sig}}), (Q_{\mathsf{pk}}, \widetilde{Q}_{\mathsf{sig}}) \right\rangle \subset (E_{\mathsf{pk}} \times E_{\mathsf{sig}})[2^a].$$
4. Compute the $(2^a, 2^a)$-isogeny
$$\Phi : E_{\mathsf{pk}} \times E_{\mathsf{sig}} \to E_1 \times E_2$$
with kernel K. If no such Φ exists, output reject.
5. Compute
$$(P', _) \leftarrow \Phi((P_{\mathsf{pk}}, 0_{E_{\mathsf{sig}}})), \qquad (Q', _) \leftarrow \Phi((Q_{\mathsf{pk}}, 0_{E_{\mathsf{sig}}})).$$
6. Compute
$$w_{\mathsf{pk}} \leftarrow \omega_{2^a}(P_{\mathsf{pk}}, Q_{\mathsf{pk}}), \qquad w' \leftarrow \omega_{2^a}(P', Q')$$
7. Output accept if
$$w_{\mathsf{pk}} = (w')^q \qquad \text{or} \qquad w' = (w_{\mathsf{pk}})^q;$$
otherwise output reject.

B Graph-based threshold access structure

Threshold access structure distributes information among users so that any authorized subset can reconstruct a target object, whereas unauthorized subsets

cannot. In this work, we consider a graph-based realization of this paradigm, motivated by isogeny-based protocols.

We model the space of possible intermediate states as a directed graph $G = (V, \mathcal{E})$ with two distinguished vertices $E_0, E_t \in V$, where E_0 denotes the initial elliptic curve and E_t denotes the final elliptic curve. Each directed edge represents an isogeny path, and each user is assigned a collection of edges.

Our goal is to realize the threshold access structure in which any set of at least t users able to form an $E_0 \to E_t$ path.

Definition 8 (Graph-based (n, t) threshold access structure). *Fix integers $1 \leq t \leq n$. A* graph-based (n, t) threshold access structure *consists of*

- *a directed graph $G = (V, \mathcal{E})$,*
- *two distinguished vertices $E_0, E_t \in V$,*
- *for each user $i \in [n]$, a set of edges $\mathcal{E}_i \subseteq \mathcal{E}$,*

such that for every subset $U \subseteq [n]$ with $|U| \geq t$, the union $\bigcup_{i \in U} \mathcal{E}_i$ contains a directed path from E_0 to E_t, and for every subset $\tilde{U} \subset [n]$ with $|\tilde{U}| < t$, the union $\bigcup_{i \in \tilde{U}} \mathcal{E}_i$ does not contain any directed path from E_0 to E_t.

B.1 Optimal Perfect Hash Families

In this section, we introduce optimal ways of constructions of perfect hash families. In general, a *(n, m, t)-perfect hash family* $\mathcal{H}$ is a finite set of hash functions $h : A \to B$ such that $|A| = n, |B| = m$ and for any $X \subseteq A$ with $|X| = t$, there exists at least one $h \in \mathcal{H}$ such that $h|_X$ is one-to-one. The concept of a perfect hash family was introduced by Mehlhorn in [44], and has been studied for the compact storage and fast retrieval of data, such as reserved words in programming languages.

Throughout this paper, we only consider the case $m = t$, which is usually called a *minimal perfect hash family*. Thus, we use (n, t, t)-perfect hash families (a minimal perfect hash family), and call them simply *(n, t)-perfect hash family*.

Definition 9 ((n, t)-perfect hash family). *A family of functions*

$$\mathcal{H} = \{h_1, \ldots, h_N\}, \qquad h_j : [n] \to [t],$$

is called a (n, t)-perfect hash family *if for every subset $T \subseteq [n]$ with $|T| = t$, there exists some $j \in [N]$ such that the restriction*

$$h_j|_T : T \to [t]$$

is bijective.

Intuitively, a (n, t)-perfect hash family guarantees that every subset of t users can be assigned t distinct hash values by at least one hash function in the family.

The minimal size of an (n,t)-perfect hash family is important, as it determines the number of paths in the graph-based access structure. It is known that the optimal size is $\Theta(\frac{e^t}{\sqrt{t}}\log n)$ [44] apart from the existence of an efficient construction.

There exist recursive constructions [45,46] that generate a (n,t)-perfect hash family of size $O(C^{\log n}\log n)$, where C is a constant depending on t. However, these constructions are tailored to the case where $n = q^\ell$ for some positive integer q, which arises from their recursive framework, and thus are not directly applicable to arbitrary values of n. Therefore, we adopt the construction proposed in [47].

We first show the nearly optimal size of an (n,t)-perfect hash family as follows.

Lemma 2. *There exists a (n,t)-perfect hash family of size*

$$N > \frac{t^t}{t!}\log\binom{n}{t},$$

which is asymptotically $O(e^t\sqrt{t}\log n)$.

Proof. Fix a subset $T \subseteq [n]$ of size t. For a uniformly random function $h : [n] \to [t]$, the probability that $h|_T$ is bijective is

$$\frac{t!}{t^t}.$$

Hence, if $h_1, \ldots, h_m$ are chosen independently at random, the probability that none of them is bijective on T is at most

$$\left(1 - \frac{t!}{t^t}\right)^m \le \exp\left(-m\frac{t!}{t^t}\right).$$

Since there are $\binom{n}{t}$ possible subsets T, the union bound implies that the probability that some t-subset is not separated is at most

$$\binom{n}{t}\exp\left(-m\frac{t!}{t^t}\right).$$

Therefore, a (n,t)-perfect hash family exists whenever

$$\binom{n}{t}\exp\left(-m\frac{t!}{t^t}\right) < 1.$$

Equivalently, it suffices to take

$$m > \frac{t^t}{t!}\log\binom{n}{t}.$$

□

Although the size $O(e^t\sqrt{t}\log n)$ is known to be nearly optimal, it is not trivial to build an explicit construction; Naor, Schulman and Srinivasan proposed a deterministic construction in [47]. Instead of describing the details of the construction, we use their method as an algorithmic building block. The time complexity and the size of the generated perfect hash family of this construction are described in Theorem 3.

Theorem 3 ([47, Theorem 3 (iii)]). *A (n,t)-perfect hash family of cardinality N can be constructed in time linear in N, where $N = e^t t^{O(\log t)} \log n$. Explicitly, we denote such construction algorithm as* $\mathsf{GenHashFamily}(n,t)$ *which takes as input (n,t) and outputs a (n,t)-perfect hash family.*

For small parameters $n, t \leq 10$, we use a simpler deterministic algorithm to compute a perfect hash family, which is introduced in [45]. We describe the details of the construction in Appendix C.1.

We also need the following $\mathsf{SelectHashFunc}$ algorithm to find a proper function h_i in the family $\mathcal{H}$ for the given subset of size t.

Algorithm 8: $\mathsf{SelectHashFunc}_{n,t}(\mathcal{H}, S)$

Input: A (n,t)-perfect hash family $\mathcal{H}$ and a subset $S \subseteq [n]$ such that $|S| = t$.
Output: A hash function $h_i \in \mathcal{H}$ such that $h_i|_S$ is bijective and its index i in the canonical order of $\mathcal{H}$.

```
Sort {h_i}_{1≤i≤N} ← H in a canonical order.
for i ← 1 to N do
    if h_i(S) = [t] then
        return (h_i, i).
return ⊥
```

B.2 Order-independent compression for the adaptive model

The drawback of the previous construction is that it still depends on a suitable ordering of the users. This is insufficient for unordered settings, where users may appear in an arbitrary order.

The idea is to construct multiple paths while taking their order into account and forming a graph. We can derive the unordered graph-based threshold access structure. This ensures that the threshold signature scheme operates correctly regardless of the order in which the parties participate. We describe the details in Appendix G.

C Detailed construction of SPRITZ

C.1 Explicit construction of an (n,t)-perfect hash family

For small parameters $n, t \leq 10$, we use the following explicit construction of an (n,t)-perfect hash family.

Lemma 3 (Theorem 2.2 in [45]). *For any $n \geq t \geq 2$, there exists an (n,t)-perfect hash family of size*

$$N = \binom{n - \lceil \frac{t}{2} \rceil}{\lfloor \frac{t}{2} \rfloor},$$

and Algorithm 9 constructs such a family deterministically.

Proof. For any subset $S = \{x_1, \ldots, x_t\} \subseteq [n]$, where $x_1 < \cdots < x_t$, let $S_{\mathsf{even}} = \{x_2, x_4, \ldots, x_{2\lfloor t/2 \rfloor}\}$. Let $h_{S_{\mathsf{even}}}$ be the function defined by

$$h_{S_{\mathsf{even}}}(x) = \begin{cases} 1 & \text{if } x < x_2 \\ 2i-1 & \text{if } x_{2(i-1)} < x < x_{2i}, \text{ for } i = 2, 3, \ldots, \lfloor \frac{t}{2} \rfloor \\ 2i & \text{if } x = x_{2i} \text{ for } i = 1, 2, \ldots, \lfloor \frac{t}{2} \rfloor \\ t & \text{if } x > x_{2\lfloor t/2 \rfloor} \end{cases}. \tag{1}$$

Since $h_{S_{\mathsf{even}}}(x_i) = i$ for $i = 1, 2, \ldots, t$, $h_{S_{\mathsf{even}}}$ is perfect on S. Thus, the family of functions induced by all admissible even-indexed tuples is a perfect hash family.

Observe that $h_{S_{\mathsf{even}}}$ depends only on the values $x_2, x_4, \ldots, x_{2\lfloor t/2 \rfloor}$. For even t, write $w = t/2$. We have $\{x_2, x_4, \ldots, x_{2w}\} \subseteq \{2, \ldots, n\}$ and $x_{2i} \geq x_{2(i-1)} + 2$. Defining $d_i = x_{2i} - i$ gives w distinct elements $1 \leq d_1 < \cdots < d_w \leq n - w$. Thus,

$$N = \binom{n - \frac{t}{2}}{\frac{t}{2}}.$$

When t is odd, the same argument leaves one element after $x_{2\lfloor t/2 \rfloor}$ and gives $\binom{n-\lceil t/2 \rceil}{\lfloor t/2 \rfloor}$ choices. The resulting construction is given in Algorithm 9.

Algorithm 9: Deterministic $\mathsf{GenHashFamily}(n,t)$

Input: Positive integers n, t such that $2 \leq t \leq n$.
Output: A canonically ordered (n,t)-perfect hash family $\mathcal{H}$.

Set $\mathcal{H} \leftarrow ()$.
for *each subset* $X \subseteq [n - \lceil \frac{t}{2} \rceil]$ *such that* $|X| = \lfloor \frac{t}{2} \rfloor$, *in lexicographic order* **do**
  Write $d_1 < \cdots < d_{\lfloor \frac{t}{2} \rfloor}$ for the ordered elements of X.
  Set $x_{2i} = d_i + i$ for all $1 \leq i \leq \lfloor \frac{t}{2} \rfloor$.
  Define the family member h_X by

$$h_X(x) = \begin{cases} 1 & \text{if } x < x_2 \\ 2i-1 & \text{if } x_{2(i-1)} < x < x_{2i}, \text{ for } i = 2, 3, \ldots, \lfloor \frac{t}{2} \rfloor \\ 2i & \text{if } x = x_{2i} \text{ for } i = 1, 2, \ldots, \lfloor \frac{t}{2} \rfloor \\ t & \text{if } x > x_{2\lfloor t/2 \rfloor} \end{cases}.$$

  Append h_X to $\mathcal{H}$.
return $\mathcal{H}$.

For the small-parameter implementation, one may derive $h_{S_{\mathsf{even}}}$ and its index in the canonical enumeration directly from Equation 1, rather than searching the entire stored family. The key-generation and signing algorithms must use the same canonical enumeration.

C.2 KLPT algorithm

The algorithm relies on several subroutines introduced in [9], except for FullRepresentInteger, which was introduced in [48]. The subroutine FullRepresentInteger is conjectured to output well-distributed elements in $\mathcal{O}_0$.

- $\mathsf{FullRepresentInteger}(M)$: given $M \in \mathbb{N}$ with $M > p$, finds $\alpha \in \mathcal{O}_0$ of norm M.
- $\mathsf{IdealModConstraint}(I, \alpha)$: given a left $\mathcal{O}_0$-ideal I of norm N and $\alpha \in \mathcal{O}_0$ with $\mathrm{nrd}(\alpha) = Nn$ and $\gcd(n, N) = 1$, finds $(C_0 : D_0) \in \mathbb{P}^1(\mathbb{Z}/N\mathbb{Z})$ such that $\alpha j(C_0 + \omega D_0) \in I$.
- $\mathsf{StrongApproximation}_n(N, C, D)$: given a prime N and $C, D \in \mathbb{Z}$, finds $\mu = z\mu_0 + N\mu_1 \in \mathcal{O}_0$ of norm n, with $z \in \mathbb{Z}$, $\mu_0 = j(C + \omega D)$, and $\mu_1 \in \mathcal{O}_0$.

For $\beta \in I$, write $\chi_I(\beta) := I\overline{\beta}/\mathrm{nrd}(I)$ for the standard equivalent-ideal transformation.

Algorithm 10: $\mathsf{KLPT}_3(I, m)$

Input: A left $\mathcal{O}_0$-ideal I of prime norm $N = \mathrm{nrd}(I) \neq 3$ such that $N < p^{1/4}$, and a positive integer m such that $3^m > 3p^2N^2$.
Output: A left $\mathcal{O}_0$-ideal J of norm 3^m such that $I \sim J$.

Set $e \leftarrow \lfloor \log_3(p/N) \rfloor + 1$.
$\alpha \leftarrow \mathsf{FullRepresentInteger}(N \cdot 3^e)$.
$(C : D) \leftarrow \mathsf{IdealModConstraint}(I, \alpha)$.
Choose integer lifts of C, D and retain the same notation.
$\mu \leftarrow \mathsf{StrongApproximation}_{3^{m-e}}(N, C, D)$.
$\beta \leftarrow \alpha \cdot \mu$.
return $\chi_I(\beta)$.

Indeed, $p/N < 3^e \leq 3p/N$, so the stated bound implies $3^{m-e} > pN^3$, the success condition for StrongApproximation. Moreover, $\alpha\mu_0 \in I$ and $N\mathcal{O}_0 \subseteq I$ imply $\beta = \alpha\mu \in I$, while $\mathrm{nrd}(\beta) = N3^m$. Thus, $\chi_I(\beta)$ is well defined, has norm 3^m, and is equivalent to I.

C.3 Our variant of the Qlapoti algorithm

The Qlapoti algorithm [28] was originally developed for SQIsign parameters, where the prime has the form $p = 2^a f - 1$. In this setting, the algorithm constructs a $(2^a, 2^a)$-isogeny containing the target ideal and relies on $2^a \approx p$.

Our threshold setting introduces an additional 3^b factor. Accordingly, we use a variant that constructs a $(2^a 3^b, 2^a 3^b)$-isogeny containing the target ideal; the relevant parameter condition is $2^a 3^b \approx p$.

Implementing this variant of IdealToIsogeny requires an efficient procedure for computing $(3^b, 3^b)$-isogenies. We use the method based on fast Kummer surfaces from [49].

C.4 The hash function H_{prime}

Algorithm 6 uses a hash function $H_{\mathsf{prime}} : \{0,1\}^* \to \mathsf{Primes}_a^{\mathsf{TS}}$, where $\mathsf{Primes}_a^{\mathsf{TS}}$ is the dual-prime set defined in Section 3. This construction follows PRISM-sig [10, Section 3.3], with the additional requirement that $2^a - q$ is also prime.

Let $H_{a-2} : \{0,1\}^* \to [0, 2^{a-2})$ be a cryptographic hash function which evaluates the PRISM hash function for $\sqrt{a} \approx \sqrt{2\lambda}$ times (because of Appendix C.6). Given input data, we define $H_{\mathsf{prime}}(\mathsf{data})$ by repeatedly computing

$$q = 2^{a-1} + 2 \cdot H_{a-2}(\mathsf{data} \,\|\, \mathsf{counter}) + 1,$$

where counter is incremented from 1 until both q and $2^a - q$ are prime, and $q \in (2^{a-1}, 3 \cdot 2^{a-2})$.

The prescribed interval is exactly the interval in $\mathsf{Primes}_a^{\mathsf{TS}}$. Requiring both factors to be large primes prevents the signing degree $q(2^a - q)$ from decomposing into many small torsion factors, as discussed in Section E.1. The search is deterministic, so the signer and verifier derive the same value from $\mathsf{data} = \mathsf{pk} \| m$; the internal counter is suppressed in the algorithms.

C.5 Parameter choices

We give a detailed analysis of our parameter choices in Appendix C.6.

Let $p = 2^a \cdot 3^b \cdot f - 1$ be a prime. The value of $a \approx 2\lambda$ is determined to satisfy the collision resistance of H_{prime} and for the weak-relation heuristic (Definition 4) to be applied. The value of $b \approx 2\lambda \log_3(2)$ is chosen against the MITM attack.

Given a pair (a, b), the factor f is chosen as the smallest integer that makes p prime. Then, we have following p values:

NIST level	λ	a	b	p
I	128	251	162	$2^{252} \cdot 3^{162} \cdot 23 - 1$
III	192	378	243	$2^{379} \cdot 3^{243} \cdot 103 - 1$
V	256	506	323	$2^{507} \cdot 3^{323} \cdot 319 - 1$

For these values of p, we have the signature size $|\mathsf{sig}| = 2\lceil \log_2 p \rceil + 3a$ bits, and the public key size $|\mathsf{pk}| = 2\lceil \log_2 p \rceil$ bits.

Consequently, we have the following signature and public key sizes of compressed versions:

NIST level	λ	pk(bytes)	sig(bytes)
I	128	129	223
III	192	193	335
V	256	257	447

C.6 Evidence on parameter choices

The number of integers $q \leq 2^a$ such that both q and $2^a - q$ are prime is, according to the Hardy–Littlewood conjecture [50, Conjecture A], asymptotically

$$2C_2 \frac{2^a}{a^2} \prod_{\substack{p \mid 2^a \\ p>2}} \frac{p-1}{p-2},$$

where $C_2 \approx 0.66$ is the twin-prime constant. Since 2^a has no odd prime divisors, the product term is trivial, and the quantity is approximately

$$2C_2 \frac{2^a}{a^2}.$$

This reduces the search space of q by the factor of $a = 2\lambda$. From the perspective of collision resistance, the security loss compared to PRISM is approximately $\log(\sqrt{2\lambda})$ bits. To compensate for this loss, we adopt the slow-hash technique used in SQIsign [51]. Concretely, our hash function H_{a-2} evaluate the hash function of PRISM for $\sqrt{a} \approx \sqrt{2\lambda}$ iterations.

According to [10, Equation (6)], collision resistance of H_{prime} requires

$$2^\lambda \lessapprox 2^{\frac{a-2}{2}} \sqrt{a}.$$

Taking the smallest integer that satisfies this inequality gives $a \approx 2\lambda$; for example, NIST level I uses $a = 251$.

The current construction chooses b so that $2^a = \Theta(\sqrt{p})$, as required by the weak-relation uniqueness heuristic in Section 3.

Our signature is the two-dimensional representation of an isogeny of degree $q(2^a - q)$ from the public curve E_{pk}. The encoding method of PRISM-sig [10] therefore applies. The representation uses the two curves $E_{\mathsf{pk}}, E'_{\mathsf{pk}}$ and the basis images $((P_{\mathsf{pk}}, Q_{\mathsf{pk}}), (P'_{\mathsf{pk}}, Q'_{\mathsf{pk}}))$. Because E_{pk} and its canonical basis $(P_{\mathsf{pk}}, Q_{\mathsf{pk}})$ are public, the signature need only encode E'_{pk} and $(P'_{\mathsf{pk}}, Q'_{\mathsf{pk}})$. Using pairing-based point compression [10, Section 5.2], the compressed size of the final signature is

$$|\mathsf{sig}| = 2\lceil \log_2 p \rceil + 3a \quad \text{bits.}$$

The intermediate partial signatures and NIZK proofs are protocol messages and are not included in this final signature encoding.

For the final public verification key, the 2^a-basis $(P_{\mathsf{pk}}, Q_{\mathsf{pk}})$ can be chosen canonically, so the curve E_{pk} suffices. In the Montgomery model, its size is

$$|\mathsf{pk}| = 2\lceil \log_2 p \rceil \quad \text{bits.}$$

D Implementation of the proof system

D.1 Non-interactive proof of σ_0

In order to ensure that the signature is generated honestly, the first party must prove that

$$\sigma_0 = \mathsf{InitSig}(q; \mathsf{crs}),$$

where the $q = H_{\mathsf{prime}}(m)$ and the common reference string crs are publicly known. As there is no secret involved, it does not require a zero-knowledge proof. Instead, each party need to compute and check the followings:

$$\begin{aligned}
&\theta = \mathsf{FullRepresentInteger}_{E_0}(q(2^a - q) \cdot 3^b; \mathsf{crs}),\\
&P' = \theta(P_0) \text{ and } Q' = \theta(Q_0),\\
&\eta : E_0 \to E_A \text{ with kernel } \ker(\widehat{\theta}) \cap E_0[3^b],\\
&K_1 = ([-q]P_0, [3^{-b}]\eta(P')) \text{ and } K_2 = ([-q]Q_0, [3^{-b}]\eta(Q')),
\end{aligned}$$

where P_0, Q_0 are the canonical basis of $E_0[2^a]$ and (K_1, K_2) is the kernel representation of the two-dimensional isogeny associated to σ_0. The main bottleneck of this verification process is the 3^b-isogeny evaluations for $\eta(P')$ and $\eta(Q')$.

D.2 Optimal R1CS constraint of $\mathcal{R}_{\mathsf{weak\text{-}push}}$

For the public statement $(\mathsf{Com}, q, \sigma_{\mathsf{prev}}, \sigma)$ and witness μ, the relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ checks the following constraints:

1. $\mathsf{Com} = H_{\mathsf{key}}(\mu)$.
2. $j(\mathrm{Im}(\sigma_1)) = j(\mathrm{Im}(\varphi'))$ and $j(\mathrm{Im}(\sigma)) = j(\mathrm{Im}(\varphi''))$,
   where $\ker\varphi' = \langle \sigma_{\mathsf{prev},1}(R) + [\mu]\sigma_{\mathsf{prev},1}(S)\rangle$ and $\ker\varphi'' = \langle \sigma_{\mathsf{prev}}(R) + [\mu]\sigma_{\mathsf{prev}}(S)\rangle$ for the canonical basis (R, S) of the domain curve of σ_{prev}

Checks involving only q, $\sigma_{\mathsf{prev}} = \sigma_{\mathsf{prev},1} \circ \sigma_{\mathsf{prev},2}$, and $\sigma = \sigma_1 \circ \sigma_2$ are performed directly by the verifier. Only the witness-dependent conditions need to be arithmetized in R1CS.

The first constraint. To compute the hash of μ, we apply the Poseidon2 hash function [52] which is a R1CS-friendly hash function. We require collision resistance and preimage resistance on the key-generation distribution. The Poseidon2 permutation consists of the full rounds and the partial rounds. We denote by R_F the number of full rounds and by R_P the number of partial rounds.

Let

$$\mathbf{x} = (x_0, \ldots, x_{t-1}) \in \mathbb{F}_p^t$$

be the internal state of Poseidon2, where t is the state width. In a full round, the S-box is applied to every state element:

$$(x_0, \ldots, x_{t-1}) \longmapsto (x_0^\alpha, \ldots, x_{t-1}^\alpha).$$

Consequently, each full round contains t S-box evaluations.

In a partial round, the S-box is applied only to the first state element:

$$(x_0, x_1, \ldots, x_{t-1}) \longmapsto (x_0^\alpha, x_1, \ldots, x_{t-1}).$$

Thus, each partial round contains only one S-box evaluation.

The full rounds are divided equally between the beginning and the end of the permutation, while the partial rounds are placed in the middle. The resulting round structure is

$$\mathsf{Full}^{R_F/2} \circ \mathsf{Partial}^{R_P} \circ \mathsf{Full}^{R_F/2}.$$

The full rounds provide rapid diffusion and resistance against statistical and algebraic attacks. Partial rounds reduce the arithmetization cost because the nonlinear S-box is evaluated on only one state element. Therefore, the values of R_F and R_P determine the trade-off between security and proof-generation efficiency. These values are selected according to the Poseidon2 security inequalities, taking into account the field size, the state width t, the S-box degree α, and the target security level λ [52].

Since a full round contains t S-box evaluations and a partial round contains one S-box evaluation, one Poseidon2 permutation contains

$$N_{\mathsf{Sbox}} = tR_F + R_P$$

S-box evaluations.

We instantiate Poseidon2 with

$$t = 3, \qquad \alpha = 3, \qquad R_F = 8.$$

It follows that

$$N_{\mathsf{Sbox}} = 3R_F + R_P = 24 + R_P.$$

For example, at the $\lambda = 128$ security level, we use

$$R_F = 8, \qquad R_P = 83.$$

Therefore, one permutation contains

$$N_{\mathsf{Sbox}} = 3 \cdot 8 + 83 = 107$$

cubic S-box evaluations.

Our implementation uses the cubic S-box

$$S(x) = x^3.$$

It is evaluated as

$$x_2 = x \cdot x, \qquad x_3 = x_2 \cdot x,$$

and therefore requires two R1CS multiplication constraints.

The external and internal linear layers of our Poseidon2 instantiation consist only of additions and multiplications by constants. They can therefore be

Table 4: Poseidon2 parameters and the number of $\mathbb{F}_p$-constraints for the SPRITZ parameter sets. The column n_λ denotes $\lceil \log_2 p \rceil$ for each security parameter λ. R_F is the number of full rounds and R_P is the number of partial rounds. N_{Sbox} is the number of S-box evaluations in one permutation. C_{perm} is the number of R1CS $\mathbb{F}_p$-constraints for one permutation.

λ	n_λ	R_F	R_P	N_{Sbox}	C_{perm}
128	514	8	83	107	214
192	771	8	127	151	302
256	1028	8	170	194	388

represented as linear combinations and do not introduce additional nonlinear multiplication constraints.

Consequently, the R1CS cost of one nonconstant Poseidon2 permutation is

$$C_{\mathsf{perm}} = 2N_{\mathsf{Sbox}} = 2(tR_F + R_P).$$

For our $t = 3$ and $R_F = 8$ instantiation, this becomes

$$C_{\mathsf{perm}} = 2(3R_F + R_P) = 2(24 + R_P).$$

At the $\lambda = 128$ security level, this gives

$$C_{\mathsf{perm}} = 2(24 + 83) = 214$$

R1CS multiplication constraints per nonconstant permutation. Table 4 summarizes the parameters and costs for the SPRITZ parameter sets.

The second constraint. As the codomain curves of $\sigma_{\mathsf{prev},1}$ and σ_{prev} are public, the only constraints to be verified are the codomain curves of φ' and φ''. These require two 3^b-isogeny computations and extra point additions and scalar multiplications to recover the kernel points of φ' and φ'' from μ. According to the result in [32], two 3^b-isogeny codomain computation requires $20b$ $\mathbb{F}_p$-constraints.

D.3 NIZK candidate using Spartan and WHIR

In the benchmark test in Section 4, we implement the zero-knowledge argument of SPRITZ based on Spartan [53] and a hash-based WHIR commitment backend [54].

Spartan is a family of general-purpose zkSNARKs for R1CS that combines multilinear representations, sumcheck, and polynomial commitments. Since its original implementation uses a polynomial commitment scheme based on the discrete logarithm assumption, achieving post-quantum security requires replacing it with a post-quantum alternative. WHIR is an interactive oracle proof of proximity (IOPP) for constrained Reed–Solomon codes. It also yields a hash-based polynomial commitment scheme (PCS) that can be combined with Spartan as

a post-quantum PCS. WHIR requires a finite field $\mathbb{F}$ whose multiplicative group has order divisible by a sufficiently large power of two. For the SPRITZ prime $p = 2^a 3^b f - 1$, we use $\mathbb{F} = \mathbb{F}_{p^2}$, whose multiplicative group has order $p^2 - 1$, divisible by 2^{a+1}. Our WHIR implementation is conservatively parameterized in the unique-decoding regime, relying on proven proximity-gap bounds rather than conjectured bounds near list-decoding capacity [55]. Consequently, recent counterexamples to the list-decoding conjecture [56,57] do not invalidate the proximity-gap bounds underlying our parameter choices.

The zero-knowledge layer follows the masking and constrained code reduction framework of Chiesa, Fenzi, and Weissenberg [58]. Merkle authentication and the Fiat-Shamir transcript use SHA-512, and the proof backend requires no secret setup trapdoor. This architecture targets post-quantum security through hash-based IOP compilation in the quantum random oracle model [59,60].

E Security analysis of SPRITZ

E.1 Hardness assumption of SPRITZ

In this section, we assume that n and t are chosen as $\mathsf{poly}(\lambda)$.

Definition 10 (SPRITZ hardness assumption). *Let λ be a security parameter. For any quantum polynomial time adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$, we run the following game*

$$\begin{aligned}
(n, t) &\leftarrow \mathcal{A}_1(\lambda) \\
(p, \mathcal{H}, \mathsf{crs}_{\mathsf{init}}) &\leftarrow \mathsf{Setup}(\lambda, n, t) \\
(\{v_i\}_{1 \le i \le |\mathcal{H}|}, \mathsf{state}) &\leftarrow \mathcal{A}_2(p, n, t, \mathcal{H}, \mathsf{crs}_{\mathsf{init}}) \\
(\{\mathsf{pk}_i\}_{1 \le i \le n}, \mathsf{pk}, \{\mathsf{sk}_i\}_{1 \le i \le n}) &\leftarrow \mathsf{KeyGen}(p, n, t, \mathcal{H}) \\
\{\varphi_{i,j}\}_{1 \le i \le |\mathcal{H}|, 1 \le j \le t} &\leftarrow \{\mathsf{sk}_i\}_{1 \le i \le n} \\
\sigma &\leftarrow \mathcal{A}_3^{\mathsf{PUSH}_{i,v_i}}(\{\mathsf{pk}_i\}_{1 \le i \le n}, \mathsf{pk}, \{\varphi_{i,j}\}_{j \ne v_i}, \mathsf{state}),
\end{aligned}$$

where $\mathrm{domain}(\sigma) \cong \mathsf{pk}$ *and* PUSH_{i,v_i} *is the push-forward oracle associated to the secret φ_{i,v_i}. It only takes as input an isogeny σ_{v_i} such that $\sigma_{v_i} = [\varphi_{i,1}]_* [\varphi_{i,2}]_* \dots [\varphi_{i,v_i-1}]_* \sigma_0$ and $\sigma_0 \leftarrow \mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$ for $q \in \mathsf{Primes}_a^{\mathsf{TS}}$. We restrict to parameter families for which the protocol runs in polynomial time and $|\mathcal{H}| = \mathrm{poly}(\lambda)$. Each PUSH_{i,v_i} oracle returns $\perp$ on inadmissible inputs and records degrees only for admissible queries. Let $\mathcal{Q}_{\mathsf{PUSH}}$ be the set of degrees recorded by the oracles PUSH_{i,v_i}. Then we have*

$$\Pr[\deg(\sigma) = q(2^a - q) : q \in \mathsf{Primes}_a^{\mathsf{TS}} \text{ and } q(2^a - q) \notin \mathcal{Q}_{\mathsf{PUSH}}] = \mathsf{negl}(\lambda).$$

Theorem 4. *Assuming the SPRITZ hardness assumption (Definition 10), the heuristic assumption of Definition 4, collision resistance of H_{prime} and H_{key}, and that the NIZK satisfies Definition 7 SPRITZ satisfies correctness and is secure against quantum adversaries in the static TS-UF-0 threshold security model (Definition 6).*

Proof. Correctness. Key generation supplies a chain from E_0 to pk, and $\mathsf{InitSig}$ produces a cyclic degree-$q(2^a-q)$ isogeny σ_0 from E_0. Since $\gcd(q(2^a-q), 3^b) = 1$, correctness of push-forward implies inductively that

$$\sigma_r = [\varphi_{\ell,r}]_*\sigma_{r-1}, \qquad r = 1, \ldots, t,$$

is cyclic of degree $q(2^a - q)$.

Every honest push-forward satisfies $R_{\mathsf{weak\text{-}push}}$ with its committed partial secrets, and all preceding push-forwards are checked. By NIZK completeness and a union bound over polynomially many proof checks, all honest signing steps complete except with negligible probability. Finally, $\mathrm{dom}(\sigma_t) \cong \mathsf{pk}$, and the two-dimensional representation produced by $\mathsf{PushForwardSig}$ satisfies the splitting and pairing checks of Algorithm 7. Thus the final signature σ_t is accepted with probability $1 - \mathsf{negl}(\lambda)$.

Static TS-UF-0 threshold security. Let $\mathcal{B}$ be a QPT adversary in the static TS-UF-0 game. For each hash function h_i, define the canonical chain by

$$\tau_{i,0}(q) = \mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}}), \qquad \tau_{i,j}(q) = [\varphi_{i,j}]_*\tau_{i,j-1}(q).$$

We first show that every prefix accepted by an honest signer is a canonical chain, except with negligible probability. Consider the first incorrect push-forward in such a prefix. Adaptive knowledge soundness (Definition 7) extracts a valid witness for its statement of $\mathcal{R}_{\mathsf{weak\text{-}push}}$. A witness differing from the real secret μ yields a collision in H_{key}. Otherwise, since the preceding push-forward is canonical, the heuristic assumption of Definition 4 excludes the incorrect push-forward results with overwhelming probability. A union bound over polynomially many checked push-forwards therefore bounds the probability of a noncanonical accepted prefix by a negligible function.

We use a hybrid argument. Let G_0 be the real signing game, and let G_1 be the same game except that it aborts at the first accepted noncanonical prefix. G_0 and G_1 are identical except that the first noncanonical accepted prefix occurs, which is negligible. Now, in G_1, all accepted prefixes are canonical before it aborts. In G_2, replace all honest proofs in G_1 by the outputs of the stateful simulator from Definition 7.

In G_1, all honest proof requests have valid witnesses. Adaptive multi-theorem zero knowledge (Definition 7) permits replacing their proofs by the outputs of one stateful simulator, with negligible change in the adversary's success probability in G_2.

We now construct an adversary $\mathcal{A}$ for Definition 10. After running $\mathcal{B}$'s corruption phase, $\mathcal{A}_2$ chooses

$$v_i \in [t] \setminus h_i(C) \qquad \text{for every } i \in [|\mathcal{H}|],$$

which is possible because $|C| < t$. The challenge consequently reveals all corrupted shares. The reduction computes push-forwards along revealed $\varphi_{i,j}$'s itself and uses PUSH_{i,v_i} for hidden edges. For the game G_2, it generates honest proofs with the simulator above, maintaining the same oracle and simulator

state throughout. Since all of the oracle responses are admissible, the reduction preserves $\mathcal{B}$'s forging probability up to a negligible term.

Then, each admissible PUSH_{i,v_i} query completes a non-aborting honest partial-signing response. Its message therefore belongs to the signing game's recorded message set $\mathcal{Q}_{\mathsf{sign}}$ (the set $\mathcal{Q}$ in Definition 6). For a fresh forgery $(m^\star, \sigma^\star)$, collision resistance of H_{prime} implies that $q^\star = H_{\mathsf{prime}}(\mathsf{pk} \| m^\star)$ differs from the hash output of every such recorded message, except with negligible probability. It therefore solves the SPRITZ hard problem of Definition 10. □

E.2 Security analysis of the SPRITZ hard problem

E.2.1 Attacks on PRISM's SPEDIO problem In Definition 10, from the algorithm $\mathcal{A}_3$'s perspective, a naive way to solve the problem is to use the oracles PUSH_{i,v_i} and isogenies $\{\varphi_{i,j}\}_{j \neq v_i}$ as subroutines only for SPEDIO used in PRISM [10, Definition 5].

Definition 11 (SPEDIO). *A special degree isogeny oracle (SPEDIO) is an oracle which takes as input a supersingular elliptic curve E over $\mathbb{F}_{p^2}$ and a prime $q \in \mathsf{Primes}_a$, and returns a uniformly random cyclic isogeny of degree $q(2^a - q)$ from E.*

In [10, Problem 2], the authors assume that, given a SPEDIO, computing an isogeny of degree $q'(2^a - q')$ with $q' \in \mathsf{Primes}_a$, whose degree differs from all degrees previously generated by the oracle, is computationally hard. In [10, Proposition 2], the security of PRISM relies on this assumption.

Indeed, $\mathcal{A}_3$ obtains more than one SPEDIO oracles from PUSH_{i,v_i}'s and $\varphi_{i,j}$'s. The distributions of each derived SPEDIO oracles come from the distribution of the output of the InitSig algorithm which is assumed to be uniform. If those SPEDIO oracle outputs are independent from the adversary's view, more than one SPEDIO oracles are equivalently thought of as just one SPEDIO oracle which outputs a uniformly random isogeny of prescribed degree.

Since $\mathsf{Primes}_a^{\mathsf{TS}} \subset \mathsf{Primes}_a$, if $\mathcal{A}_3$ solves the SPEDIO problem in the SPRITZ setting, it also can break the security of PRISM, which implies that such attacks are infeasible assuming the security of PRISM.

The key point is that, from the PPT adversary's perspective, the provided SPEDIO oracles are independent each other. We make this assumption because no efficiently verifiable algebraic relation among these SPEDIOs is currently known.

E.2.2 Attacks on the push-forward oracles An adversary can try to recover the partial secrets associated to each partial signing oracle. Such key-recovery attacks are prevented because of the use of NIZK protocol and the large prime degree sampled from $\mathsf{Primes}_a^{\mathsf{TS}}$. The attack proposed in [30] is prevented by verifying the weak relation $\mathcal{R}_{\mathsf{weak\text{-}push}}$ as explained in Section 3.2.

For a large prime q, it is known that q-torsion points are only represented over a large field extension $\mathbb{F}_{p^r}$ where r has approximately the size of q ([61]). As

both q and $(2^a - q)$ are prime $\approx 2^a$, it is infeasible to extract torsion information of the secret isogeny φ associated to the push-forward oracle.

F Tree secret sharing for large threshold

One major limitation of our graph-based threshold access structure is that the size of the hash family grows exponentially with the threshold parameter t. Although this overhead is acceptable for small instances (e.g., $n < 10$), it quickly becomes prohibitive as t increases. To address this issue, we instead adopt a recursive graph structure similar to the tree-based construction proposed in [62], which avoids this exponential growth.

F.1 Reducing the number of partial secrets using the tree secret sharing

Suppose that $\mathsf{SS}(\ell, s)$ denotes a secret sharing scheme which takes as input a set of edges of a graph G, and outputs a set of edges as secret shares for s-out-of-ℓ graph-based threshold access structure.

Algorithm 11: $\mathsf{TreeSS}_L(\mathsf{sk})$ [62, Algorithm 1]

Input: secret sharing scheme for s-out-of-ℓ threshold access structure $\mathsf{SS}(\ell, s)$ where $s \geq 3$, and $\ell = 2s - 1$. Secret key sk, and the iteration number L.

Output: The set of secret shares $\{\mathsf{share}_j\}_{j \in [\ell^L]}$.

$(\mathsf{share}_1^{(1)}, \ldots, \mathsf{share}_\ell^{(1)}) \leftarrow \mathsf{SS}(\ell, s)(\mathsf{sk})$.
for $i \leftarrow 2..L$ **do**
  for $j \leftarrow 1..\ell^{i-1}$ **do**
    $(\mathsf{share}^{(i)}_{\ell(j-1)+1}, \ldots, \mathsf{share}^{(i)}_{\ell(j-1)+\ell}) \leftarrow \mathsf{SS}(\ell, s)(\mathsf{share}_j^{(i-1)})$.
return A set of secret shares $\{\mathsf{share}_i^{(L)}\}_{i \in [\ell^L]}$.

Algorithm 11 describes the graph-based variant of the TreeSS algorithm from [62]. Since the secret sharing scheme SS takes a set of edges as input and produces shares that are themselves sets of edges, the original TreeSS construction extends naturally to our graph-based threshold setting.

Theorem 5 (Theorem 4.6 in [62]). *Let N be an odd number and s be a small positive integer such that $s \ll N$, and $\ell = 2s - 1$. Given a graph-based secret sharing scheme $\mathsf{SS}(\ell, s)$ and the iteration number $L \geq \log_{c_s} N + \log_s N + O(1)$ where $c_s = \frac{\ell}{2^{\ell-1}} \cdot \binom{\ell-1}{s-1}$, there exists $\frac{N+1}{2}$-out-of-N threshold secret sharing scheme with at least $1/2$ success probability and the number of secret shares is*

$$\ell^L = O(N^{\log_{c_s} \ell + \log_s \ell}).$$

According to Section 4.4 in [62], we can generate a t-out-of-n threshold access structure for arbitrary t using a $\frac{N+1}{2}$-out-of-N threshold scheme. In this case,

the number of shared secrets is still $O(N^{\log_{c_s} \ell + \log_s \ell})$ for constant integers s and $\ell = 2s - 1$.

Figure 2 illustrates the recursive construction of the threshold access structure from the case $t = 2$. In the leftmost graph, reaching the final node E_t from E_0 requires traversing two edges, corresponding to the threshold access structure with parameters $(n, t) = (3, 2)$. The middle graph represents the recursive expansion of each single edge, where reaching E_t from E_0 again requires traversing at least four edges. This gives us a $(n, 4)$-threshold structure for some n. Applying the same recursive expansion once more yields a $(n, 8)$-threshold access structure for some n. By adding some virtual parties, these structures can be used as a (n, t)-threshold access structure for an arbitrary $t \leq 8$.

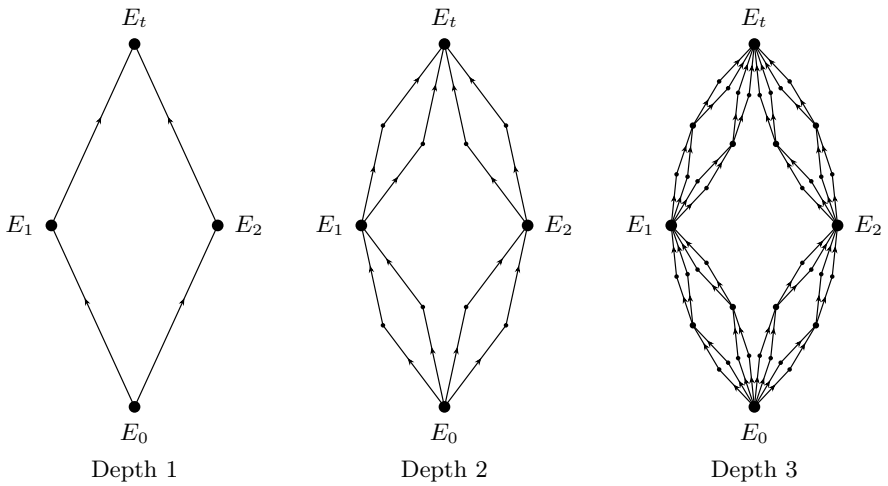


Fig. 2: An example of the recursive construction of the threshold access structure.

F.2 Constructing larger SPRITZ from small (n_s, t_s)-SPRITZ

We construct an (n, t)-SPRITZ by recursively replacing every secret isogeny edge with a graph-based (n_s, t_s)-threshold access structure. The output shares of each replacement are again sets of secret isogeny edges and can therefore be expanded recursively, via tree secret sharing. In order to construct the recursive graph illustrated in Figure 2, we need a generalized LongIdealToIsogeny algorithm that converts an ideal to its corresponding isogeny when the input ideal does not necessarily start from $\mathcal{O}_0$.

Algorithm 12: GeneralLongIdealToIsogeny(I_{prev}, J)

Input: A left $\mathcal{O}_0$-ideal I_{prev} with a left $\mathcal{O}_R(I_{\mathsf{prev}})$-ideal J of norm 3^{bt} such that the corresponding isogeny ϕ_J is cyclic.

Output: The kernel representations of φ_i's corresponding to J_i's, where $J = J_1 \cdots J_t$ such that $\mathrm{nrd}(J_i) = 3^b$.

Let $\mathcal{O} = \mathcal{O}_R(I_{\mathsf{prev}})$.
Set $\mathcal{T} \leftarrow \varnothing$.
for $\mathsf{step} \leftarrow 0 \dots t-1$ **do**
  Compute $L \leftarrow \mathsf{EquivalentPrimeIdeal}(I_{\mathsf{prev}} \cdot (J + 3^{\mathsf{step}\cdot b} \cdot \mathcal{O}))$.
  Compute $\psi_L \leftarrow \mathsf{IdealToIsogeny}(L)$. // using Qlapoti algorithm, Appendix C.3
  Compute the 3^b-torsion cyclic subgroup $G = E_0[L \cap J'] \cap E_0[3^b]$, where $J' = (J + 3^{\mathsf{step}\cdot b} \cdot \mathcal{O})^{-1} \cdot (J + 3^{(\mathsf{step}+1)b} \cdot \mathcal{O})$.
  Compute $\varphi_{\mathsf{step}+1}$ such that $\ker \varphi_{\mathsf{step}+1} = \psi_L(G)$.
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\varphi_{\mathsf{step}+1}\}$.
return the kernel representations of $\mathcal{T} = \{\varphi_i\}_{1 \le i \le t}$.

During the recursive expansion, an edge is represented by

$$D = (I_{\mathsf{prev}}, J),$$

where I_{prev} is a left $\mathcal{O}_0$-ideal such that $\mathcal{O}_R(I_{\mathsf{prev}}) \cong \mathcal{O}_L(J)$.

By one step of the expansion, we obtain N_s equivalent chains of edges with the same starting and ending nodes as D, as follows.

$$\begin{aligned} D_1 &= \{(I_{\mathsf{prev}} \times J_{1,1} \times \dots J_{1,i-1}, J_{1,i})\}_{1 \le i \le t_s}, \\ &\vdots \\ D_{N_s} &= \{(I_{\mathsf{prev}} \times J_{N_s,1} \times \dots J_{N_s,i-1}, J_{N_s,i})\}_{1 \le i \le t_s}, \end{aligned}$$

where N_s is the size of a (n_s, t_s)-perfect hash family, $\mathcal{O}_R(J_{i,j}) \cong \mathcal{O}_L(J_{i,j+1})$, $\mathcal{O}_L(J_{i,1}) \cong \mathcal{O}_R(I_{\mathsf{prev}})$, and all $\mathcal{O}_R(J_{i,t_s})$'s are the same up to isomorphism. Let us denote the set χ_i of edges given by a (n_s, t_s)-perfect hash family as

$$\chi_1, \dots, \chi_{n_s} \in \mathcal{P}(\{D_{i,j}\}_{1 \le i \le N_s, 1 \le j \le t_s}),$$

where $\chi_1, \dots, \chi_{n_s}$ induces a (n_s, t_s)-threshold access structure.

At the end of the recursive expansion, by applying the KLPT_3 algorithm, we obtain edges $D = (I_{\mathsf{prev}}, J)$ such that $\mathrm{nrd}(J) = 3^{t_s b}$. Thus, Algorithm 12 gives us the corresponding isogenies:

$$\{\varphi_i\}_{1 \le i \le t_s} \leftarrow \mathsf{GeneralLongIdealToIsogeny}(I_{\mathsf{prev}}, J).$$

Algorithm 13 shows the corresponding process.

Algorithm 13: $\mathrm{Expand}_{N,t}(I_{\mathsf{prev}}, J)$

Input: An edge $D = (I_{\mathsf{prev}}, J)$ where I_{prev} is a left $\mathcal{O}_0$-ideal such that $\mathcal{O}_R(I_{\mathsf{prev}}) \cong \mathcal{O}_L(J)$.
Output: A set of $t \cdot N$ edges $\{D_{i,j} = (I_{\mathsf{prev}} \times J_{i,1} \times \cdots \times J_{i,j-1}, J_{i,j})\}_{1 \le i \le N, 1 \le j \le t}$.

Compute $I' \leftarrow \mathsf{EquivalentPrimeIdeal}(I_{\mathsf{prev}})$.
for $i = 1...N$ **do**
  Compute $J'_i \leftarrow \mathsf{KLPT}_3([I']^* J)$. // Algorithm 10
  for $j = 1...t$ **do**
    Compute $J_i \leftarrow [I']_* J'_i$.
    Factoring J_i to obtain $J_{i,j}$ such that $J_i = J_{i,1} \times \cdots \times J_{i,t}$ and $\mathrm{nrd}(J_{i,j}) = 3^b$.
    Set $D_{i,j} \leftarrow (I_{\mathsf{prev}} \times J_{i,1} \times \cdots \times J_{i,j-1}, J_{i,j})$.
return $\{D_{i,j}\}_{1 \le i \le N, 1 \le j \le t}$

Setup. We first convert the (n', t')-threshold access structure into a (n, t)-threshold access structure using $r_{\mathrm{on}} + r_{\mathrm{off}}$ virtual parties by setting $r_{\mathrm{on}} = t' - t$, $r_{\mathrm{off}} = n' - n - r_{\mathrm{on}}$ when n' and t' are much larger than n and t. The r_{on} virtual parties are always available, whereas the r_{off} virtual parties are always unavailable.

Algorithm 14: $\mathsf{Setup}_{\mathsf{Tree}}(\lambda, n, t, n_s, t_s)$

Input: A security parameter λ and integers $2 \le t \le n$.
Output: Public parameters pp.

Compute $(p, \mathcal{H}, \mathsf{crs}_{\mathsf{init}}) \leftarrow \mathsf{Setup}(\lambda, n, t)$; // Algorithm 1
Set $r_{\mathrm{on}} \leftarrow \max\{n + 1 - 2t, 0\}$, $r_{\mathrm{off}} \leftarrow \max\{2t - n - 1, 0\}$.
Set $n' \leftarrow n + r_{\mathrm{on}} + r_{\mathrm{off}}$, $t' \leftarrow t + r_{\mathrm{on}} = \frac{n'+1}{2}$.
Set $\mathcal{A} \leftarrow \{n+1, \ldots, n + r_{\mathrm{on}}\}$ and $\mathcal{D} \leftarrow \{n + r_{\mathrm{on}} + 1, \ldots, n'\}$.
Set $\alpha_0 \leftarrow \frac{t'}{n'}$, $\alpha_{\ell+1} \leftarrow \sum_{k=t_s}^{n_s} \binom{n_s}{k} \cdot \alpha_\ell^k \cdot (1 - \alpha_\ell)^{n_s - k}$.
Choose the smallest $L \ge 1$ such that $1 - \alpha_L \le 2^{-\lambda}$.
for $z \in [n_s]^L$ **do**
  Sample $\rho(z) \xleftarrow{\$} [n']$.
Set $\mathsf{pp} := ((p, \mathcal{H}, \mathsf{crs}_{\mathsf{init}}), (n, t), (n', t'), (n_s, t_s), L, \rho, \mathcal{A}, \mathcal{D})$.
return pp

Correctness of $\mathsf{Setup}_{\mathsf{Tree}}$.

Definition 12. *For an always-available set $S \subseteq [n]$, we define the availability of a node $z \in [n_s]^{\leq L}$ recursively by*

$$\mathfrak{A}_S(z) := \begin{cases} 1, & \text{if } |z| = L \text{ and } \rho(z) \in S \cup \mathcal{A}, \\ 0, & \text{if } |z| = L \text{ and } \rho(z) \notin S \cup \mathcal{A}, \\ 1, & \text{if } |z| < L \text{ and } \sum_{i=1}^{n_s} \mathfrak{A}_S(z||i) \geq t_s, \\ 0, & \text{if } |z| < L \text{ and } \sum_{i=1}^{n_s} \mathfrak{A}_S(z||i) < t_s. \end{cases}$$

Except with probability at most $2^{-\lambda}$ over Algorithm 14, the following threshold property holds for every $S \subseteq [n]$:

$$\bigcup_{i \in S} \mathsf{sk}_i \text{ contains a directed path from } E_0 \text{ to } E_{\mathsf{pk}} \quad \Longleftrightarrow \quad |S| \geq t.$$

To prove the claim, first fix S with $|S| = t$. Then, exactly $|S| + r_{\text{on}} = t + r_{\text{on}} = t'$ parties are available including r_{on} virtual parties. Since each leaf owner is independently sampled from $[n']$, a leaf is available with probability $\alpha_0 = \frac{t'}{n'} > \frac{1}{2}$. An internal node is available if and only if at least t_s of its n_s children are available. Hence its availability probability satisfies

$$\alpha_{\ell+1} = \sum_{k=t_s}^{n_s} \binom{n_s}{k} \cdot \alpha_\ell^k \cdot (1 - \alpha_\ell)^{(n_s - k)}.$$

where α_ℓ denotes the availability probability at level $(L - \ell)$ of the tree structure.

Key generation. The formal key generation algorithm is described in Algorithm 15. We denote the root vertex by ϵ. In this algorithm, for every generated vertex z, we let $\mathcal{X}_z$ denote the set of graph paths assigned to z, which is immediately computed from known graph paths. We also define $d_z := |\mathcal{X}_z|$, which is the number of graph paths assigned to the leaf vertex z.

Algorithm 15: $\mathsf{KeyGen}_{\mathsf{Tree}}(\mathsf{pp})$

Input: Public parameters
$\mathsf{pp} = ((p, \mathcal{H}, \mathsf{crs}_{\mathsf{init}}), (n, t), (n', t'), (n_s, t_s), L, \rho, \mathcal{A}, \mathcal{D})$.
Output: A public key pk, partial secret keys $(\mathsf{sk}_i)_{i\in[n]}$, and partial public keys $(\mathsf{pk}_i)_{i\in[n]}$.

Choose a random left $\mathcal{O}_0$-ideal I such that $\mathrm{nrd}(I) \approx p^{1/4}$.
Set the initial edge $\chi_\varepsilon \leftarrow \{(\mathcal{O}_0, I)\}$, where ε denotes the empty string.
for $d \leftarrow 1...L$ **do**
  for $\chi_x \in \{\chi_x\}_{|x|=d-1}$ **do**
    Set $\mathsf{sec}_i \leftarrow \varnothing$ for $1 \le i \le n_s$.
    for $D \in \chi_x$ **do**
      $\{D_{i,j}\}_{1\le i\le N_s, 1\le j\le t_s} \leftarrow \mathsf{Expand}_{N_s,t_s}(D)$.
      Grouping $\{\mathsf{sec}'_i\}_{1\le i\le n_s} \leftarrow \{D_{i,j}\}_{1\le i\le N_s,1\le j\le t_s}$ according to the (n_s, t_s)-perfect hash family $\mathcal{H}$.
      Set $\mathsf{sec}_i \leftarrow \mathsf{sec}_i \cup \mathsf{sec}'_i$ for $1 \le i \le n_s$.
    $\chi_{x||i} \leftarrow \mathsf{sec}_i$ for $1 \le i \le n_s$.
for $x \in [n_s]^L$ **do**
  Set $\mathsf{SingleKey}_x \leftarrow \varnothing$.
  for $D \in \chi_x$ **do**
    $\mathsf{SingleKey}_x \leftarrow \mathsf{SingleKey}_x \cup \mathsf{GeneralLongIdealToIsogeny}(D)$.
  Set $\mathsf{Com}_x \leftarrow \varnothing$.
  for $\phi \in \mathsf{SingleKey}_x$ **do**
    $\mathsf{Com}_x \leftarrow \mathsf{Com}_x \cup \{(H_{\mathsf{key}}(\mu), \mathrm{Im}(\phi))\}$, where $\ker\phi = \langle R + [\mu]S\rangle$ and (R, S) is the canonical 3^b-torsion basis of domain(ϕ).
Set pk as the last curve in the graph.
for $i \in [n]$ **do**
  Set $\mathsf{sk}_i \leftarrow \{\mathsf{SingleKey}_x : x \in [n_s]^L, \text{ and } (\rho(x) = i \text{ or } \rho(x) \in \mathcal{A})\}$.
  Set $\mathsf{pk}_i \leftarrow \{\mathsf{Com}_x : \mathsf{SingleKey}_x \in \mathsf{sk}_i\}$
return $\left(\mathsf{pk}, (\mathsf{sk}_i)_{i\in[n]}, (\mathsf{pk}_i)_{i\in[n]}\right)$.

All paths contained in the same leaf set χ_x are assigned together to the single virtual party $\rho(x)$. If $\rho(x) \in \mathcal{A}$, the entire leaf set is included in every party's partial key. If $\rho(x) \in \mathcal{D}$, it is included in no partial key.

The total number of 3^b-isogenies generated in $\mathsf{KeyGen}_{\mathsf{Tree}}$ is $(N_s t_s)^L$. By Theorem 5, the required minimum depth is $L = O(\log_c n + \log_{t_s} n)$, where $c = \frac{n_s}{2^{n_s-1}}\binom{n_s-1}{t_s-1}$. Thus, the total number of 3^b-isogenies is $\mathsf{poly}(n)$. Consequently, the sizes of all shared secrets are also $\mathsf{poly}(n)$.

Signing and verification. The tree-based construction uses the initialization, partial signature verification, push-forward, and NIZK procedures of Algorithm 6 and the verification algorithm (Algorithm 7) without modification. Only the perfect-hash-based path selection is replaced by the following deterministic tree-path selection. In the tree-based construction, the $\mathsf{SelectHashFunc}$ algorithm in

Algorithm 6 selects a path of (n_s, t_s)-threshold access structure which corresponds to the received prefix.

G Construction of adaptive SPRITZ

This appendix extends the threshold construction to online signer selection with an arbitrary arrival order. We first make the graph layer order-free and then certify every partial signatures with the same NIZK relation.

G.1 Order-free access structure for the adaptive model

By augmenting a perfect hash family with all permutations of its output range, we obtain an order-independent threshold access structure: any t users can form a path in their order of arrival. If the underlying family has $O(\sqrt{t}e^t \log n)$ functions, the augmented family has $O(t!\sqrt{t}e^t \log n)$ functions.

We first define an order-free (n, t)-perfect hash family as follows.

Definition 13 (Order-free (n, t)-perfect hash family). *A (n, t)-perfect hash family*

$$\mathcal{H} = \{h_1, \ldots, h_m\}, \qquad h_j : [n] \to [t],$$

is called an order-free (n, t)-perfect hash family if for every ordered t-tuple

$$(u_1, \ldots, u_t)$$

of pairwise distinct elements of $[n]$, there exists some $h \in \mathcal{H}$ such that

$$h(u_i) = i \qquad \text{for all } i \in [t].$$

For an ordered prefix $\mathbf{u} = (u_1, \ldots, u_k)$ with $1 \le k \le t$, it is convenient to write

$$\mathcal{H}[\mathbf{u}] := \{\, h \in \mathcal{H} : h(u_r) = r \text{ for all } r \in [k] \,\}.$$

In an order-free (n, t)-perfect hash family, we then have

$$\mathcal{H}[\mathbf{u}] \neq \varnothing$$

for every ordered prefix of length at most t. For the empty prefix, we set $\mathcal{H}[()] := \mathcal{H}$.

Every h_j indexes a chain of length t. A function remains compatible with the prefix $(u_1, \ldots, u_k)$ precisely when it assigns u_r to the r-th edge for every $r \le k \le t$. For every fixed ordered t-tuple, at least one hash function h_j is compatible with the fixed tuple.

We now define the order-free graph-based (n, t) threshold access structure induced by an order-free (n, t)-perfect hash family. Let

$$\mathcal{H} = \{h_1, \ldots, h_m\}, \qquad h_j : [n] \to [t],$$

be an order-free (n, t)-perfect hash family. Define a directed graph $G = (V, \mathcal{E})$ by

$$V := \{E_0, E_t\} \cup \{v_{j,k} : j \in [m],\ 1 \le k \le t-1\},$$

and

$$\mathcal{E} := \{\varphi_{j,k} : v_{j,k-1} \to v_{j,k} \mid j \in [m],\ k \in [t]\},$$

where $v_{j,0} := E_0$ and $v_{j,t} := E_t$. For each user $u \in [n]$, assign the edge set

$$\mathcal{E}_u := \{\varphi_{j,h_j(u)} : v_{j,h_j(u)-1} \to v_{j,h_j(u)} \mid j \in [m]\} \subseteq \mathcal{E}.$$

Remark 3 (Size). Let N_0 be the size of the underlying (n, t)-perfect hash family. Composing every function with all permutations of $[t]$ produces at most $t!N_0$ functions, and each function indexes one length-t chain. Thus, the number of chains and the size of each per-user edge table in the construction of Section B.1 grow by at most a factor of $t!$, while each chain still has length t. Hence the number of paths is

$$N = O(t! \cdot \sqrt{t} e^t \log n).$$

Each user receives one edge for every hash function in $\mathcal{H}$. For cryptographic instantiations, we restrict to parameters for which $|\mathcal{H}|$ and the total number of edges are polynomial in λ. In the intended applications, n and t are small. For the case that n and t are large, we use the tree-based structure like Section F.

G.2 Algorithms for adaptive SPRITZ

In threshold protocols, the complete signing set need not be online at the same time. We therefore let an ordered prefix of distinct participating parties grow one party at a time and retain every hash-family row compatible with that prefix. Here, "adaptive" refers only to online signer selection and arrival order, not to adaptive corruption.

The signing procedure in Algorithm 6 selects a hash function after the subset S of participants is known. In the adaptive variant, the compatible set of hash functions shrinks monotonically as the prefix grows, and each surviving hash function carries a chain of partial signatures. This increases the partial secret keys, the auxiliary partial public keys, and the intermediate signing work during the communication between parties. It does not change the SPRITZ public key pk or the final signature returned directly by the last signer.

G.2.1 Setup and key generation The setup uses an order-free (n, t)-perfect hash family rather than an ordinary (n, t)-perfect hash family.

An order-free (n, t)-perfect hash family can be obtained by composing every function in an ordinary (n, t)-perfect hash family with every permutation of $[t]$. Hence we may use the construction in Theorem 3 or Appendix C.1 as the underlying family. Algorithm 16 returns exactly the hash functions compatible with a given ordered prefix.

Algorithm 16: $\mathsf{SelectMultipleHashFuncs}_{n,t}(\mathcal{H}, S)$

Input: An order-free (n,t)-perfect hash family $\mathcal{H}$ and an ordered tuple $S = (u_1, \ldots, u_r) \in [n]^*$ such that $0 \le r \le t$.
Output: A set of labeled hash functions $\{(h_i, i)\}_i$.

if *the entries of* S *are not pairwise distinct* **then return** $\perp$;
Set $N \leftarrow |\mathcal{H}|$ and list $\mathcal{H} = (h_1, \ldots, h_N)$ in canonical order.
Set $B \leftarrow \emptyset$.
for $i \leftarrow 1$ *to* N **do**
  if $h_i(u_j) = j$ *for every* $j \in [r]$ **then**
    Set $B \leftarrow B \cup \{(h_i, i)\}$.
return B.

Algorithm 17: $\mathsf{AdaptiveSetup}(\lambda, n, t)$

Input: A security parameter λ and positive integers n, t such that $2 \le t \le n$.
Output: Public parameters $\mathsf{pp} = (1^\lambda, \mathsf{par}, n, t, \mathcal{H}, \mathsf{crs}_{\mathsf{init}})$, where $\mathsf{par} = (p, a, b)$.

Compute $\mathcal{H}_0 \leftarrow \mathsf{GenHashFamily}(n, t)$ and $\widetilde{\mathcal{H}} \leftarrow \{\pi \circ h : h \in \mathcal{H}_0,\ \pi \in \mathrm{Sym}([t])\}$.
Remove duplicate functions and write $\mathcal{H} = (h_1, \ldots, h_N)$ in canonical order.
Compute (p, a, b) according to Appendix C.6, and set $\mathsf{par} = (p, a, b)$.
Sample an independent public random string $\mathsf{crs}_{\mathsf{init}}$ for $\mathsf{InitSig}$.
return $\mathsf{pp} = (1^\lambda, \mathsf{par}, n, t, \mathcal{H}, \mathsf{crs}_{\mathsf{init}})$.

The setup algorithm (Algorithm 1) is modified only in its choice of hash family, as shown in Algorithm 17.

Using the security parameter and $(\mathsf{par}, n, t, \mathcal{H})$ recorded in pp, adaptive key generation applies Algorithm 3 to the enlarged family $\mathcal{H}$. Its implicit security parameter is the λ in pp, and the value of b used to generate the keys must equal the value fixed in par. After generating $\{\varphi_{i,j}\}_{(i,j)\in[N]\times[t]}$, it explicitly forms the complete commitment table

$$\mathsf{Com}_{i,j} := H_{\mathsf{key}}(\mu_{i,j}) \qquad ((i,j) \in [N] \times [t]).$$

For $\mathcal{H} = (h_1, \ldots, h_N)$, it returns the global key pk and, for every party u, the row-indexed values

$$\mathsf{pk}_u[i] = (\mathsf{Com}_{i,h_i(u)}, E_{i,h_i(u)}), \qquad \mathsf{sk}_u[i] = \varphi_{i,h_i(u)} \quad (i \in [N]),$$

where $E_{i,j} = \mathrm{Im}(\varphi_{i,j})$. The value $\mathsf{Com}_{i,h_i(u)}$ is the public commitment used in the NIZK statement. All later algorithms use the same canonical row ordering fixed by setup.

G.2.2 Signing and verification We now extend the SPRITZ signing algorithm (Algorithm 6) to make it compatible with every ordered prefix of signers. Fix $\mathsf{pp} = (1^\lambda, \mathsf{par}, n, t, \mathcal{H}, \mathsf{crs}_{\mathsf{init}})$. For $S_r = (u_1, \ldots, u_r)$, let

$$B_r := \mathsf{SelectMultipleHashFuncs}_{n,t}(\mathcal{H}, S_r), \qquad I_r := \{i : (h_i, i) \in B_r\}.$$

The sets B_r (and hence I_r) are nested as r increases and are nonempty for every ordered prefix of distinct parties of length at most t.

For $1 \le r < t$ and $(h_i, i) \in B_r$, define the set of proofs

$$\Gamma_{i,r} := \left(\{\sigma_{i,j}\}_{0 \le j \le r}, \{\pi^{\mathsf{sig}}_{i,j}\}_{1 \le j \le r} \right), \qquad \Sigma_{S_r} := \{(i, \Gamma_{i,r}) : i \in I_r\}.$$

We set $\Sigma_{S_0} := \emptyset$ for the empty prefix. The algorithm

$$\mathsf{VerifyAdaptivePrefix}(\mathsf{pp}, \mathsf{PK}, S_r, m, \Sigma_{S_r})$$

returns 1 at $r = 0$ exactly for this empty state. For $1 \le r < t$, it returns 1 exactly when all of the following checks pass:

1. $S_r \in [n]^*$ has pairwise distinct entries, $q = H_{\mathsf{prime}}(\mathsf{pk} \| m)$ is successfully computed in $\mathsf{Primes}_a^{\mathsf{TS}}$, and B_r is well defined.
2. Σ_{S_r} has a unique canonical parsing with exactly one record for every $i \in I_r$ and no other records, and each $\Gamma_{i,r}$ contains exactly $r+1$ isogenies and r proofs.
3. for every $i \in I_r$, $\sigma_{i,0}$ is the valid cyclic $q(2^a - q)$-isogeny $\mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$.
4. for every $i \in I_r$ and $j \in [r]$, the entry $\mathsf{pk}_{u_j}[i] = (\mathsf{Com}_{i,j}, E_{i,j})$ exists, and

$$\mathsf{NIZK}_{\mathcal{R}_{\mathsf{weak\text{-}push}}}.\mathsf{Verify}^{H_{\mathsf{nizk}}} \left(1^\lambda, (\mathsf{Com}_{i,j}, q, \sigma_{i,j-1}, \sigma_{i,j}), \pi^{\mathsf{sig}}_{i,j} \right) = 1.$$

Here $h_i(u_j) = j$, so $E_{i,j}$ is the codomain curve committed at key generation. The NIZK verifies the secret-dependent part of $\mathcal{R}_{\mathsf{weak\text{-}push}}$.

Algorithm 18 shows the signing algorithm of the adaptive SPRITZ. Since adaptive SPRITZ differs from the original scheme only in its choice of hash family, its correctness and security follow by the same arguments as in Appendix E.

Algorithm 18: $\mathsf{AdaptiveSign}_k(\mathsf{pp}, \mathsf{PK}, S_{\mathsf{prev}}, \Sigma_{S_{\mathsf{prev}}}, m, \mathsf{sk}_k)$

Input: Public parameters pp, public signing material $\mathsf{PK} = (\mathsf{pk}, \{\mathsf{pk}_u\}_{u\in[n]})$, an ordered prefix S_{prev}, its signatures and proofs $\Sigma_{S_{\mathsf{prev}}}$, a message m, and party k's key sk_k.

Output: If the new prefix has length less than t, the extended prefix S and the signatures and proofs Σ_S; if it has length t, a SPRITZ signature σ; otherwise, $\perp$.

Parse $(1^\lambda, (p, a, b), n, t, \mathcal{H}, (\mathsf{crs}_{\mathsf{init}}, \mathsf{crs}_{\mathsf{nizk}})) \leftarrow \mathsf{pp}$.
if $S_{\mathsf{prev}} \notin [n]^*$ *or its entries are not pairwise distinct* **then return** $\perp$;
if $k \notin [n]$, $|S_{\mathsf{prev}}| \geq t$, *or* $k \in S_{\mathsf{prev}}$ **then return** $\perp$;
Set $S \leftarrow S_{\mathsf{prev}} \| k$ and $r \leftarrow |S|$.
Compute $B_r \leftarrow \mathsf{SelectMultipleHashFuncs}_{n,t}(\mathcal{H}, S)$.
if $B_r \in \{\perp, \emptyset\}$ **then return** $\perp$;
Compute $q \leftarrow H_{\mathsf{prime}}(\mathsf{pk} \| m)$.
if $r = 1$ **then**
    if $\Sigma_{S_{\mathsf{prev}}} \neq \emptyset$ **then return** $\perp$;
    Compute $\sigma_0 \leftarrow \mathsf{InitSig}(q; \mathsf{crs}_{\mathsf{init}})$.
else
    if $\mathsf{VerifyAdaptivePrefix}(\mathsf{pp}, \mathsf{PK}, S_{\mathsf{prev}}, m, \Sigma_{S_{\mathsf{prev}}}) \neq 1$ **then return** $\perp$;
Set $\widetilde{B}_r \leftarrow B_r$ and, if $r < t$, $\Sigma_S \leftarrow \emptyset$.
if $r = t$ **then**
    Set $i^* \leftarrow \min\{i : (h_i, i) \in B_r\}$ and $\widetilde{B}_r \leftarrow \{(h_{i^*}, i^*)\}$.
for $(h_i, i) \in \widetilde{B}_r$ **do**
    if $r = 1$ **then**
        Set $\sigma_{i,0} \leftarrow \sigma_0$ and $\Gamma_{i,0} \leftarrow (\{\sigma_{i,0}\}, \emptyset)$.
    else
        Load $\Gamma_{i,r-1}$ from $\Sigma_{S_{\mathsf{prev}}}$.
    Set $\psi_{i,r} \leftarrow \mathsf{sk}_k[i]$ and $(\mathsf{Com}_{i,r}, E_{i,r}) \leftarrow \mathsf{pk}_k[i]$.
    if $\psi_{i,r}$ *is not a valid cyclic* 3^b*-isogeny with* $\mathrm{dom}(\psi_{i,r}) \cong \mathrm{dom}(\sigma_{i,r-1})$ **then return** $\perp$;
    if $H_{\mathsf{key}}(\mathrm{encode}(\ker \psi_{i,r})) \neq \mathsf{Com}_{i,r}$ **then return** $\perp$;
    Compute $\sigma_{i,r} \leftarrow \mathsf{PushForwardSig}(\sigma_{i,r-1}, \psi_{i,r})$.
    if $\sigma_{i,r} = \perp$ *or* $\sigma_{i,r}$ *is not a valid cyclic* $q(2^a - q)$*-isogeny* **then return** $\perp$;
    if $r = t$ **then**
        return $\sigma_{i^*,t}$.
    Set $x^{\mathsf{sig}}_{i,r} \leftarrow (\mathsf{Com}_{i,r}, q, \sigma_{i,r-1}, \sigma_{i,r})$.
    Compute $\pi^{\mathsf{sig}}_{i,r} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{weak\text{-}push}}}.\mathsf{Prove}^{H_{\mathsf{nizk}}}(1^\lambda, x^{\mathsf{sig}}_{i,r}, \psi_{i,r})$.
    Append $(i, \Gamma_{i,r})$ to Σ_S, where $\Gamma_{i,r}$ extends $\Gamma_{i,r-1}$ by $(\sigma_{i,r}, \pi^{\mathsf{sig}}_{i,r})$.
return (S, Σ_S).